Tema 2
JavaScript



De dónde viene JavaScript, cómo se prueba lo primero que se escribe con él, y el vocabulario básico (variables, tipos, conversión, operadores y comentarios) con el que se construye cualquier programa.

1. Fundamentos de JavaScript

1.1 Introducción

JavaScript es, hoy en día, uno de los lenguajes de programación más utilizados del mundo, por tres razones muy concretas: es útil (resuelve problemas reales de interacción con el usuario), es práctico (no requiere instalación ni compilación previa) y está disponible en cualquier navegador web, en cualquier sistema operativo, sin que el usuario tenga que hacer nada especial para ejecutarlo.

Un poco de historia
JavaScript fue creado por Brendan Eich en 1995, mientras trabajaba en Netscape Communications, la empresa responsable de uno de los primeros navegadores web populares. El lenguaje se diseñó y se implementó en muy pocos días, con un objetivo modesto: añadir pequeñas dosis de interactividad a páginas web que, hasta entonces, eran completamente estáticas.

Durante su desarrollo pasó por varios nombres: primero se llamó Mocha, después LiveScript, y finalmente se rebautizó como JavaScript poco antes de su lanzamiento (una decisión de mercado, aprovechando la popularidad que tenía Java en ese momento, aunque ambos lenguajes son técnicamente muy distintos entre sí y no tienen relación directa). Lo que nació como un lenguaje sencillo y secundario ha terminado convirtiéndose en el lenguaje de programación que se ejecuta en prácticamente todos los sitios web del mundo.

Características principales

- **Ligero**: su sintaxis y su motor de ejecución están pensados para ser rápidos de cargar e interpretar en el navegador.
- **Multiplataforma**: se ejecuta igual en Windows, Linux o macOS, y en cualquier navegador que lo implemente, sin necesidad de adaptar el código.
- **Basado en prototipos**: a diferencia de lenguajes como Java o C#, JavaScript no usa clases de forma nativa para la herencia, sino un mecanismo llamado *prototipos*.
- **Orientado a objetos y basado en eventos**: casi todo en JavaScript es un objeto, y gran parte de la interacción con la página se construye reaccionando a eventos (un clic, una tecla pulsada, una carga completada).
- **Interpretado**: no requiere un paso de compilación previo por parte del desarrollador antes de ejecutarse (como ya se vio en el Tema 1, el motor del navegador lo procesa internamente, pero eso es transparente para quien programa).
- **Débilmente tipado** (tipado dinámico): una misma variable puede almacenar valores de distintos tipos a lo largo de la ejecución, sin que haga falta declarar de antemano qué tipo de dato va a contener.

1.2 Primer contacto: Hola Mundo

Antes de entrar en la teoría del lenguaje, conviene ver JavaScript funcionando. Existen varias formas de mostrar un mensaje al usuario o de comprobar que un script se está ejecutando correctamente.

La función alert
Muestra una ventana emergente del propio navegador (no forma parte de la página) con un mensaje y un botón de aceptar. Detiene la ejecución del script hasta que el usuario la cierra.

```
alert('Hola mundo');
```

El método write del objeto document
Escribe texto directamente en el documento HTML, en el punto donde se ejecuta el script.

```
document.write('Hola mundo');
```

Importante: document.write() se enseña aquí porque aparece con frecuencia en tutoriales antiguos, pero está desaconsejado en desarrollo profesional: si se llama después de que la página haya terminado de cargar, borra por completo el contenido del documento y lo sustituye por el texto escrito. Se muestra únicamente con fines didácticos, para reconocerlo si aparece en código de otros.

El objeto console
Envía el mensaje a la consola de las herramientas de desarrollador del navegador, sin interrumpir ni modificar la página. Es la herramienta que se usará de forma constante a lo largo de todo el curso, tanto para mostrar resultados como para depurar código.

```
console.log('Hola mundo');
```

Nota: De las tres opciones, console.log() es la que se usará por defecto en el resto del curso: no interrumpe la ejecución con una ventana emergente, no modifica el documento, y es la herramienta estándar de depuración en cualquier entorno de desarrollo profesional.

Actividad · Hola Mundo

Crea tu primer fichero .html con un <script> dentro de la etiqueta <body> que muestre "Hola mundo". Prueba, uno por uno, los tres métodos vistos (alert, document.write y console.log) y observa las diferencias.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Hola mundo</title>
</head>
<body>
    <h1>Mi primer script</h1>

    <script>
        console.log('Hola mundo');
```

1.3 La etiqueta noscript

A veces el navegador no puede interpretar JavaScript: bien porque el navegador no lo soporta, bien porque el propio usuario lo ha desactivado desde su configuración. En esos casos, la página se carga igualmente, pero todo el comportamiento que dependía de JavaScript deja de funcionar, sin ningún aviso.

Por esa razón, es buena práctica avisar de que la página necesita JavaScript para funcionar correctamente, usando la etiqueta <noscript>. Su contenido solo se muestra cuando el navegador no ejecuta JavaScript; en caso contrario, se ignora por completo.

```
<noscript>
  <p>La página que estás viendo requiere JavaScript para su correcto funcionamiento.</p>
</noscript>
```

1.4 Funciones de interacción: prompt y confirm

JavaScript incluye tres funciones integradas que muestran una pequeña ventana del navegador para comunicarse con el usuario. Ya se ha visto alert, aquí se completan las otras dos, muy útiles para empezar a experimentar con datos que introduce el propio usuario.

prompt(mensaje, valorPorDefecto)
Muestra un mensaje, un campo de texto y dos botones (aceptar y cancelar). Devuelve el texto introducido como string, o null si el usuario pulsa "Cancelar".

```
const nombreCliente = prompt('¿Cuál es tu nombre?', '');


if (nombreCliente === null) {
  console.log('El usuario ha cancelado el diálogo');
} else {
  console.log(`Hola, ${nombreCliente}`);
}
```

confirm(pregunta)
Muestra un mensaje y dos botones (aceptar y cancelar). Devuelve true o false según el botón pulsado.

```
const confirmado = confirm('¿Seguro que quieres eliminar este producto?');


if (confirmado) {
  console.log('Producto eliminado');
} else {
  console.log('Operación cancelada');
}
```

Importante: Las tres funciones son **bloqueantes (también alert)**: mientras la ventana está abierta, toda la página deja de responder, incluida cualquier otra ejecución de JavaScript. Por eso, en aplicaciones web profesionales no se usan en la interfaz final (se

sustituyen por ventanas modales propias, construidas con HTML y CSS, que no bloquean el navegador). Aquí se usan porque son la forma más rápida de leer un dato del usuario o mostrar un resultado sin necesitar todavía el DOM, que se estudiará más adelante.

1.5 Enlazar HTML con JavaScript: <script> y sus variantes

Hay dos decisiones distintas que se mezclan a menudo: dónde escribes el código (dentro del HTML o en un fichero aparte) y dónde colocas la etiqueta <script> (en el <head> o en el <body>, y con qué atributos). Te lo explico por partes.

Las tres formas de vincular el código
Inline (dentro de un atributo, tipo onclick="..."): mala práctica, mezcla HTML y JS. Se enseña para reconocerlo, no para usarlo.

Interno: el código va dentro de la propia etiqueta <script>, en el mismo HTML.

```
<script>
  console.log('Hola');
</script>
```

Externo: el código vive en un fichero .js aparte, enlazado con el atributo src. Es la forma recomendada siempre que el código no sea trivial.

```
<script src="./app.js"></script>
```

Dónde colocar la etiqueta <script> (y con qué atributo)
Aquí está el matiz que seguramente te interesa. El navegador parsea el HTML de arriba abajo, y por defecto, en cuanto encuentra un <script>, detiene ese parseo para descargar y ejecutar el script antes de seguir.

Colocación	Comportamiento	¿Cuándo usarla?
<script src="..."> en el <head>, sin atributos	Bloquea el parseo del HTML nada más encontrarlo. Si el script intenta acceder al DOM (document.querySelector(...)), falla porque el <body> todavía no existe.	Nunca, para código que toque el DOM
<script src="..."> al final del <body>, justo antes de </body>	Sigue siendo bloqueante, pero como ya está todo el HTML anterior parseado, el DOM ya existe cuando se ejecuta. Es la técnica clásica de "toda la vida".	Válida, pero superada por defer
<script defer src="..."> en el <head>	Se descarga en paralelo sin bloquear el parseo, y se ejecuta justo cuando el HTML ya está completo. Respeta el orden entre varios scripts.	La opción recomendada hoy en día
<script type="module" src="..."> en el <head>	Se comporta como defer automáticamente, además de dar módulos ES (import/export).	Cuando trabajas con módulos

Por qué defer en el <head> es hoy la mejor práctica
La idea clave: con defer, el navegador empieza a descargar el script cuanto antes (nada más leer el <head>), pero espera a ejecutarlo hasta que el HTML esté completo. Consigues lo mejor de los dos mundos: descarga temprana, ejecución segura.

```
<!DOCTYPE html>
```

```
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi página</title>
  <script defer src="./app.js"></script>
</head>
<body>
  <h1 id="titulo"></h1>
</body>
</html>
```

Si en cambio lo pones al final del <body> sin defer, el navegador ni siquiera empieza a descargarlo hasta que ha terminado de parsear todo el HTML anterior, así que en páginas grandes pierdes ese tiempo de descarga en paralelo que sí aprovechas con defer.

1.6 Sintaxis básica del lenguaje

Antes de entrar en detalle en cada elemento del lenguaje, conviene fijar unas pocas reglas generales que se van a cumplir siempre, en cualquier programa JavaScript.

Los espacios en blanco y los saltos de línea no importan
El motor de JavaScript ignora los espacios, tabulaciones y saltos de línea adicionales. Se usan únicamente para que el código sea más legible para las personas, no para el propio motor.

```
// Estas dos formas son equivalentes para el motor de JavaScript
const total = 10 + 5;


const
  total2
    =
      10 + 5;
```

Nota: Que el motor las ignore no significa que a las personas les dé igual. Un formateador como Prettier (mencionado en el Tema 1) se encarga de aplicar un espaciado consistente de forma automática, para que el equipo no tenga que discutir sobre estilo.

Es sensible a mayúsculas y minúsculas (*case sensitive*)
JavaScript distingue mayúsculas de minúsculas en todos sus identificadores: nombres de variables, funciones, propiedades y palabras clave. Dos nombres que solo se diferencien en el uso de mayúsculas son, para el lenguaje, dos cosas completamente distintas.

```
const nombreUsuario = 'ana';
const NombreUsuario = 'ANA';
```

```
console.log(nombreUsuario); // 'ana'
console.log(NombreUsuario); // 'ANA' (una variable totalmente distinta a la anterior)
```

Es un lenguaje débilmente tipado
Como se ha adelantado anteriormente, al declarar una variable no se indica de qué tipo será su valor, y ese valor puede cambiar de tipo durante la ejecución del programa.

El punto y coma es opcional, pero muy recomendable
JavaScript no obliga a terminar cada sentencia con punto y coma (;): el motor intenta insertarlos automáticamente donde cree que corresponde, mediante un mecanismo llamado *ASI* (*Automatic Semicolon Insertion*). El problema es que ese mecanismo no siempre acierta, y puede producir errores silenciosos muy difíciles de detectar.

Se pueden incluir comentarios
Con // para una sola línea, o con /* y */ para varias líneas. Más adelante se trata en profundidad, incluyendo el estándar JSDoc usado en desarrollo profesional.

1.7 Variables: declaración, inicialización y ámbito

Una variable es un espacio con nombre donde se almacena un valor que puede cambiar durante la ejecución del programa. JavaScript ofrece tres palabras clave para declarar variables: **var, let y const**. Aunque **var** sigue siendo válida y aparece en código antiguo, en desarrollo profesional actual no se utiliza.

Declaración e inicialización
Declarar una variable es crear el espacio de memoria y asociarle un nombre. **Inicializar** es asignarle un primer valor. Ambas acciones pueden hacerse por separado o a la vez.

```
// Declaración sin inicializar
let precioProducto;
console.log(precioProducto); // undefined

// Declaración con inicialización en la misma línea
let nombreProducto = 'Teclado mecánico';

// const obliga a inicializar en el momento de la declaración
const IVA = 0.21;

// Esto lanza un error de sintaxis: const exige valor inicial
const stockMinimo;
```

let vs. const: cuándo usar cada una

	let	const
¿Se puede reasignar?	Sí	No

¿Se puede declarar sin valor inicial?	Sí	No
¿Cuándo usarla?	El valor va a cambiar	El valor no se va a reasignar

```
let contadorVisitas = 0;
contadorVisitas = contadorVisitas + 1; // correcto, let permite reasignar

const nombreEmpresa = 'Acme Software';
nombreEmpresa = 'Otra empresa'; // TypeError: Assignment to constant variable.
```

Importante: La recomendación en desarrollo profesional es usar const por defecto y recurrir a let únicamente cuando el valor vaya a cambiar (por ejemplo, un contador de un bucle o un acumulador). No es solo una cuestión de estilo: declarar algo como const documenta la intención del código y evita reasignaciones accidentales, una fuente habitual de errores difíciles de rastrear.

Actividad · Calculadora de propina

Crea un script que pida al usuario, con prompt, el importe de una cuenta de restaurante y, con otro prompt, el porcentaje de propina que quiere dejar. Calcula el total y muéstralo con alert. Añade un confirm antes de mostrar el resultado, preguntando "¿Confirmas el cálculo?"; si el usuario cancela, muestra por consola "Cálculo cancelado" en su lugar.

Si definimos una variable sin poner a su izquierda, var, let o const, la crea por defecto como var, si queremos evitar esto, poner 'use strict' como primera línea del fichero .js.

Ámbito (*scope*) de las variables

El ámbito determina en qué parte del código una variable es accesible. let y const tienen ámbito de bloque (block scope): solo existen dentro de las llaves { } en las que se declaran.

```
function calcularPrecioFinal(precioBase) {
  if (precioBase > 100) {
    const descuento = precioBase * 0.1;
    console.log(descuento); // accesible: estamos dentro del bloque if
  }

  console.log(descuento); // ReferenceError: descuento is not defined
}
```

var, en cambio, tiene **ámbito de función** (*function scope*), no de bloque, lo que provoca comportamientos poco intuitivos:

```
function calcularPrecioFinalConVar(precioBase) {
  if (precioBase > 100) {
    var descuento = precioBase * 0.1;
  }
```

```
  console.log(descuento); // funciona, aunque estamos FUERA del if
}
```

Importante: Este es uno de los motivos por los que **var** se considera obsoleta: su ámbito de función hace que una variable "se escape" del bloque donde parecía estar confinada, lo cual facilita errores silenciosos en funciones largas. let y const resuelven este problema restringiendo la variable exactamente al bloque { } donde se declara.

Hoisting

JavaScript "eleva" las declaraciones de variables y funciones a la parte superior de su ámbito antes de ejecutar el código, un comportamiento llamado ***hoisting***. Sin embargo, let y const quedan en lo que se conoce como *temporal dead zone*: existen, pero no se pueden usar antes de la línea donde se declaran.

```
console.log(mensaje); // ReferenceError: Cannot access 'mensaje' before initialization
let mensaje = 'Hola';
```

Nota: No es necesario memorizar los detalles internos del *hoisting*. A efectos prácticos, la regla de oro es: declara siempre las variables al principio del bloque donde se van a usar, antes de leerlas o modificarlas. Siguiendo esa disciplina, el *hoisting* nunca da sorpresas.

1.8 Tipos de datos

JavaScript es un lenguaje de **tipado dinámico**: no se declara el tipo de una variable, sino que el tipo lo determina el valor que contiene en cada momento, y puede cambiar durante la ejecución.

```
let valor = 42; // valor es de tipo number
valor = 'cuarenta y dos'; // ahora valor es de tipo string
```

JavaScript distingue entre **tipos primitivos** y **tipos por referencia**.

Tipos primitivos

Un valor primitivo es inmutable y se compara y se copia **por valor**.

Tipo	Descripción	Ejemplo
number	Números enteros y decimales (un único tipo numérico)	42, 3.14, -7
string	Cadenas de texto	'Hola', "Hola"
boolean	Verdadero o falso	true, false
undefined	Variable declarada, pero sin valor asignado	let x;
null	Ausencia intencionada de valor	let x = null;
bigint	Enteros de precisión arbitraria	9007199254740993n
symbol	Valor único e irrepetible	Symbol('id')

No busques **int** que no está. Number se representa siempre en coma flotante de doble precisión (el estándar IEEE 754), tanto si el valor tiene decimales como si no. Es una decisión de diseño del lenguaje: simplifica el sistema de tipos a cambio de ceder algo de precisión y control fino que sí tienen lenguajes con tipos numéricos separados.

```
const edad = 34; // number
const nombre = 'Carlos'; // string
const activo = true; // boolean
let direccionEnvio; // undefined (declarada, sin valor)
const telefonoSecundario = null; // null (ausencia intencionada de valor)
```

Importante: undefined y null no son lo mismo, aunque ambos representan "ausencia de valor". undefined significa que la variable no ha sido inicializada (o que una propiedad o función no existe). null significa que alguien, deliberadamente, ha indicado que no hay valor. Como norma profesional: deja que undefined ocurra de forma natural (no lo asignes a mano) y usa null cuando quieras expresar explícitamente "aquí no hay nada" (por ejemplo, un campo fechaBaja: null en un usuario que sigue activo).

string: comillas simples, dobles y plantillas literales

En JavaScript existen tres formas de delimitar una cadena de texto: comillas simples ('...'), comillas dobles ("...") y comillas invertidas o *backticks* (`...`). Las dos primeras son funcionalmente idénticas: no hay ninguna diferencia de comportamiento entre usar una u otra, solo una cuestión de estilo. Lo importante es elegir una y mantenerla de forma consistente en todo el proyecto.

```
const simples = 'Producto añadido';
const dobles = "Producto añadido";
// Ambas líneas son equivalentes: JavaScript no distingue entre ellas
```

Nota: En desarrollo profesional es habitual fijar una única convención (normalmente comillas simples) mediante un formateador como Prettier, para que todo el equipo escriba el código de la misma forma sin tener que pensarlo.

También se usan cuando necesitamos poner, por ejemplo, comillas dobles dentro de una cadena con comillas simples.

```
let cadena = "Esto es 'importante'"
```

Concatenación

Concatenar es unir dos o más cadenas de texto en una sola, usando el operador +. Aquí veremos el problema que más quebraderos de cabeza da en código real.

```
const nombre = 'Ana';
const apellido = 'García';


console.log(nombre + ' ' + apellido); // 'Ana García'
```

Hasta aquí, todo previsible: dos strings con + dan un string más largo.

El problema: + con un número y un string no suma, concatena

El operador + tiene una regla poco intuitiva: si al menos uno de los dos operandos es un string, el otro se convierte a string, y se concatenan. No se suma nunca en ese caso.

```
console.log('5' + 3);    // '53' (no 8)
```

```
console.log(3 + '5');     // '35' (no 8)
console.log('El total es ' + 10 + 5); // 'El total es 105' (¡no 'El total es 15'!)
```

Esa última línea es la que más sorprende. JavaScript evalúa + de izquierda a derecha:

- 'El total es ' + 10 → como hay un string, concatena → 'El total es 10'
- 'El total es 10' + 5 → sigue habiendo un string, concatena otra vez → 'El total es 105'

El 5 nunca llegó a sumarse a nada; se pegó como texto al final.
¿Qué resultado da la siguiente línea?

```
console.log(2 + 8 + '5');
```

Dónde aparece este bug en la práctica
El caso más habitual, y el que probablemente más te vas a encontrar, es con datos que vienen de prompt() o de un formulario: **siempre llegan como string**, aunque parezcan un número.

```
const cantidad1 = prompt('¿Cuántas unidades del producto A?'); // el usuario escribe "3"
const cantidad2 = prompt('¿Cuántas unidades del producto B?'); // el usuario escribe "5"


console.log(cantidad1 + cantidad2); // '35', no 8
```

Aunque el usuario ha escrito números, prompt() los devuelve como texto, así que + los concatena en vez de sumarlos. Este es, probablemente, el ejemplo más realista de por qué hay que tener el problema en la cabeza.

El caso contrario: querías concatenar y te encuentras una resta
El problema también funciona al revés, y ahí -, * y / se comportan de forma totalmente distinta a +: fuerzan siempre la conversión a número, nunca concatenan.

```
console.log('10' - '4');  // 6  (número)
console.log('10' + '4');  // '104' (string, con el mismo tipo de operandos)
```

Es decir: + es el único operador aritmético con este doble comportamiento (suma o concatena, según el tipo). El resto de operadores matemáticos siempre convierten a número, sin excepción.

La forma profesional de evitar el problema: plantillas literales
Cuando lo que se busca es construir un mensaje con números incrustados (no operar con ellos), mezclar + con strings y números es exactamente donde se cuela el bug con más frecuencia, porque es fácil olvidar un paréntesis o el orden de las sumas. Las plantillas literales evitan el problema de raíz, porque cada expresión dentro de ${ } se evalúa por separado, sin encadenar sumas.

¿Para qué sirven las plantillas literales?

- **Interpolación de expresiones**: permiten insertar el valor de una variable o el resultado de una expresión directamente dentro del texto, sin concatenar con +. La sintaxis es ${ expresión }: todo lo que se escriba dentro de las llaves se evalúa como código JavaScript y el resultado se inserta en ese punto de la cadena.

- **Cadenas multilínea**: permiten que el texto ocupe varias líneas tal cual se escribe, sin unir varias cadenas con + ni usar el carácter especial de salto de línea.

Con comillas simples, mostrar un mensaje con dos variables obliga a concatenar así:

```
const producto = 'ratón inalámbrico';
const precio = 29.99;

const mensajeConcatenado = 'El producto "' + producto + '" cuesta ' + precio.toFixed(2) + ' €';
```

Con una plantilla literal, la misma línea queda así de legible:

```
const mensaje = `El producto "${producto}" cuesta ${precio.toFixed(2)} €`;
console.log(mensaje); // El producto "ratón inalámbrico" cuesta 29.99 €
```

Y para el caso de cadenas multilínea, esto ya no es posible con comillas simples o dobles (darían un error de sintaxis), pero sí con plantilla literal:

```
const plantillaEmail = `Estimado cliente,

Su pedido ha sido confirmado.
Gracias por su compra.`;
```

Solucionando el caso de la suma que al final concatena:

```
const cantidad1 = 3;
const cantidad2 = 5;

// Con +: hay que tener cuidado con el orden y los paréntesis
console.log('Total: ' + (cantidad1 + cantidad2)); // 'Total: 8' — funciona, pero el paréntesis es fácil de olvidar
console.log('Total: ' + cantidad1 + cantidad2);   // 'Total: 35' — bug clásico, sin el paréntesis

// Con plantilla literal: la suma ocurre dentro de ${ }, aislada, sin ambigüedad
console.log(`Total: ${cantidad1 + cantidad2}`); // 'Total: 8', siempre correcto
```

Importante: Las plantillas literales se delimitan siempre con comillas invertidas (la tecla situada normalmente junto a P, no con un apóstrofe). Si se usan comillas simples o dobles, ${producto} no se interpola: aparece tal cual, como texto literal.

Nota: En desarrollo profesional se prefieren las plantillas literales (comillas invertidas) frente a la concatenación con +, porque el resultado es mucho más legible cuando hay varias variables involucradas.

En cuanto a los strings, podemos usar caracteres especiales utilizando el carácter de escape "\":

\n	Una nueva línea
\t	Tabulador

\’	Comilla simple
\”	Comilla doble
\\	Barra inclinada

El operador typeof
Permite comprobar el tipo de un valor en tiempo de ejecución.

```
console.log(typeof 42); // 'number'
console.log(typeof 'Hola'); // 'string'
console.log(typeof true); // 'boolean'
console.log(typeof undefined); // 'undefined'
console.log(typeof null); // 'object' (ver nota siguiente)
console.log(typeof {}); // 'object'
console.log(typeof []); // 'object'
console.log(typeof function(){}); // 'function'
```

Importante: typeof null devuelve 'object'. Es un error histórico del propio lenguaje (arrastrado desde su primera versión, en 1995, por motivos de compatibilidad) y no se puede corregir sin romper páginas web existentes. Para comprobar si un valor es específicamente null, se compara de forma directa: valor === null.

1.9 Funciones para cadenas de texto

A veces necesitamos usar el método "**toString()**" para poder aplicar estos métodos.
.length
Número de caracteres de la cadena (es una propiedad, no un método: sin paréntesis).

```
const cad = 'hola';
console.log(cad.length); // 4
```

Concatenación: + y .concat()

```
const mensaje1 = 'Hola';
const mensaje2 = mensaje1.concat(' Mundo'); // 'Hola Mundo'
```

Nota: .concat() funciona, pero en código moderno se usa casi siempre + o, mejor aún, una plantilla literal. concat() se enseña sobre todo para reconocerlo si aparece en código de terceros.

.toUpperCase() / .toLowerCase()
Transforma todos los caracteres en mayúsculas/minúsculas

```
const mensaje = 'Hola';
console.log(mensaje.toUpperCase()); // 'HOLA'
console.log(mensaje.toLowerCase()); // 'hola'
```

.charAt(posicion)
Devuelve el carácter en esa posición. Si no existe, devuelve " (cadena vacía), no undefined.

```
const mensaje = 'Hola';
console.log(mensaje.charAt(0)); // 'H'
```

```
console.log(mensaje.charAt(2)); // 'l'
console.log(mensaje.charAt(9)); // '' (fuera de rango)
```

Nota: una cadena se puede tratar como un array de caracteres, así que mensaje[2] es equivalente a mensaje.charAt(2). La diferencia es que mensaje[9] (fuera de rango) devuelve undefined, mientras que charAt(9) devuelve ''.

.indexOf() / .lastIndexOf()
.indexOf() busca la **primera** coincidencia; .lastIndexOf(), la **última**. Ambas devuelven -1 si no encuentran nada, y funcionan igual con un carácter suelto o con una palabra completa. También funciona en Arrays.

```
const mensaje = 'Hola caracola';
console.log(mensaje.indexOf('a'));     // 3 (primera 'a')
console.log(mensaje.lastIndexOf('a')); // 12 (última 'a')
console.log(mensaje.indexOf('b'));     // -1

const frase = 'Hola que tal';
console.log(frase.indexOf('que')); // 5
```

Ejercicios · Cadenas
Ejercicio1. Crear una función que pida una cadena por teclado y muestre su tamaño.
Ejercicio2. Crea una función que pida la clave Secreta por pantalla, si la introduce correctamente aparece el mensaje "Has acertado" (también serán válidas: secreta, SECRETA, SeCrEtA...)

.substring(inicio, fin)
Extrae una porción de texto. El segundo parámetro es opcional (si se omite, corta hasta el final).

```
const mensaje = 'Hola Mundo';
console.log(mensaje.substring(2));    // 'la Mundo'
console.log(mensaje.substring(1, 8)); // 'ola Mun' (el índice final no se incluye)
```

Importante: .substring() tiene dos comportamientos que sorprenden si no se conocen:

- Un valor negativo se trata como 0: mensaje.substring(-2) da 'Hola Mundo' completo, no los dos últimos caracteres (para eso está .slice(), que sí admite negativos).
- Si fin es menor que inicio, JavaScript los intercambia automáticamente: mensaje.substring(5, 0) da el mismo resultado que mensaje.substring(0, 5).

.substr(inicio, numCaracteres)
Corta desde inicio, un número de caracteres concreto (en vez de hasta un índice final).

```
console.log('JavaScript'.substr(4, 6)); // 'Script'
console.log('Finland'.substr(3, 4));    // 'land'
```

Importante: .substr() está marcado como *legacy* (obsoleto) en la especificación del lenguaje, aunque todos los navegadores lo siguen soportando por compatibilidad. En código profesional se prefiere .slice(inicio, fin) (que admite índices negativos y no está en desuso) para conseguir el mismo resultado.

.split(separador)
Convierte una cadena en un array, cortando por el separador indicado. Es la operación inversa de .join() (que verás en la parte de arrays).

```
const mensaje = 'Hola Mundo, soy una cadena';
console.log(mensaje.split(' ')); // ['Hola', 'Mundo,', 'soy', 'una', 'cadena']


console.log('JavaScript'.split('')); // ['J','a','v','a','S','c','r','i','p','t'] (separador vacío: cada carácter)
```

.trim()
Elimina espacios en blanco al principio y al final

```
const nombreProducto = '  Teclado Mecánico RGB  ';


console.log(nombreProducto.trim()); // 'Teclado Mecánico RGB'
```

Ejercicios · Cadenas
Ejercicio 3. Función que defina una variable cadena con "hola que tal yo bien" y busque "tal" para mostrar la cadena desde "tal" al final.

.replace() / .replaceAll()
Sustituye una cadena de texto por otra. También admite expresiones regulares (que ya se verán más adelante). replace reemplaza la primera coincidencia y replaceAll todas.

```
const str = 'El script es útil. El script es interesante.';


console.log(str.replace('script', 'código'));    // 'El código es útil. El script es interesante.' (solo la 1ª)
console.log(str.replaceAll('script', 'código')); // 'El código es útil. El código es interesante.' (todas)
```

.padStart(longitud, relleno)
Rellena la cadena actual con una cadena dada (esta cadena puede repetirse varias veces), de modo que la cadena resultante alcance una longitud dada. El relleno es aplicado desde el inicio (izquierda) de la cadena actual

```
function generarReferencia(numeroPedido) {
  return `PED-${String(numeroPedido).padStart(6, '0')}`;
}


console.log(generarReferencia(42)); // 'PED-000042'
'abc'.padStart(10, "R");  // "RRRRRRRabc" (rellena con un carácter hasta llegar a 10)
'abc'.padStart(10);  // "       abc" (si no se pasa el segundo parámetro, usa el espacio en blanco)
'abc'.padStart(10, "foo");  // "foofoofabc" (también se puede rellenar con más de un carácter)
'1'.padStart(2,"0"); // "01" (este ejemplo es útil para mostrar segundos, minutos, …)
```

slice, corta un rango de elementos de la cadena y lo devuelve. Tiene 2 parámetros, la posición inicial y la posición final(no incluida). Si no se incluye la posición inicial, corta hasta el final de la cadena.

Devolver la extensión de un archivo pasado por parámetro.

```
const nombreArchivo = 'informe.final.PDF'
const punto = nombreArchivo.lastIndexOf('.');
console.log(nombreArchivo.slice(punto + 1).toLowerCase());
```

Substring es estándar y no está obsoleto. El motivo para preferir slice es de diseño: substring corrige silenciosamente argumentos incoherentes (negativos a 0, intercambio si inicio > fin), y ese "perdón" suele esconder errores de cálculo en lugar de hacerlos visibles. Con slice, un rango incorrecto produce un resultado vacío o inesperado que se detecta antes, y además funciona igual en arrays y strings, así que el equipo solo tiene que aprender una semántica.

repeat, hay una función que permite repetir una cadena un número de veces, es útil si queremos mostrar por ejemplo, tres <td></td> y en vez de hacer un for entre 1 y 3, hacemos un repeat:

```
let resultado = "<td></td>".repeat(3)
//Resultado = <td></td><td></td><td></td>
```

Ejercicios · Cadenas
Ejercicio 4. Crear una función que tenga como parámetro de entrada un DNI y le ponga un 0 o no al principio según corresponda (si tiene 7 caracteres, hay que ponerle un 0 delante).
Ejercicio5. Crea una función que muestre por pantalla cuántas palabras tiene una frase, mostrando: "Esta frase tiene X palabras".

1.10 Funciones para los números

toFixed(dígitos): devuelve el número original con tantos decimales como los indicados por el parámetro dígitos y realiza los redondeos necesarios.

```
let numero1 = 4564.34567;
numero1.toFixed(2); // 4564.35
numero1.toFixed(6); // 4564.345670
numero1.toFixed(); // 4564
```

El objeto Math

Math es distinto de String, Number o Date: no es un constructor (nunca se escribe new Math()), sino un único objeto con métodos y constantes matemáticas listos para usar.

Método / propiedad	Qué hace
Math.round(x)	Redondea al entero más cercano
Math.ceil(x)	Redondea siempre hacia arriba
Math.floor(x)	Redondea siempre hacia abajo

Método / propiedad	Qué hace
Math.trunc(x)	Elimina la parte decimal, sin redondear
Math.abs(x)	Valor absoluto
Math.pow(base, exp)	Potencia (equivalente a base ** exp)
Math.sqrt(x)	Raíz cuadrada
Math.max(...) / Math.min(...)	Máximo o mínimo de varios valores
Math.random()	Número decimal aleatorio entre 0 (incluido) y 1 (excluido)

```
console.log(Math.round(4.5)); // 5
console.log(Math.ceil(4.1)); // 5
console.log(Math.floor(4.9)); // 4
console.log(Math.max(12, 45, 3, 8)); // 45
```

Importante: Math.round() y toFixed() (visto en el apartado anterior) no son intercambiables. Math.round(precio * 100) / 100 devuelve un number redondeado a 2 decimales, listo para seguir operando con él.
precio.toFixed(2) devuelve un string formateado, listo para mostrarlo. Usa cada uno según lo que necesites a continuación: cálculo o presentación.

Generar un número entero aleatorio en un rango
Math.random() por sí solo solo da un decimal entre 0 y 1, que rara vez es útil directamente. La fórmula habitual para obtener un entero aleatorio entre dos valores (ambos incluidos) es:

```
/**
 * Genera un número entero aleatorio entre min y max, ambos incluidos.
 * @param {number} min - Valor mínimo posible.
 * @param {number} max - Valor máximo posible.
 * @returns {number} Entero aleatorio en el rango [min, max].
 */
function numeroAleatorioEntre(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

console.log(numeroAleatorioEntre(1, 6)); // simula una tirada de dado
```

1.11 Conversión de tipos de datos

JavaScript realiza conversiones de tipo de dos formas: de manera **implícita** (el propio lenguaje decide convertir un valor sin que se le pida) y de manera **explícita** (el desarrollador convierte un valor de un tipo a otro de forma deliberada).

Conversión implícita (*coerción*)
Ocurre automáticamente al operar con valores de tipos distintos. Es una de las zonas del lenguaje que genera más comportamientos inesperados si no se conoce bien.

```
console.log('5' + 3); // '53' (el + con un string concatena, no suma)
console.log('5' - 3); // 2 (el - fuerza conversión a number)
console.log('5' * '2'); // 10 (* también fuerza conversión a number)
console.log(5 + true); // 6 (true se convierte a 1)
console.log(5 + false); // 5 (false se convierte a 0)
console.log('5' == 5); // true (== convierte antes de comparar)
console.log('5' === 5); // false (=== NO convierte, compara tipo y valor)
```

Importante: El operador == (doble igual) aplica reglas de conversión que a veces producen resultados poco intuitivos (por ejemplo, '' == 0 da true, y null == undefined también da true). El operador === compara sin convertir, por lo que el resultado es siempre predecible.

Conversión explícita
El desarrollador indica de forma clara a qué tipo quiere convertir un valor, usando funciones integradas del lenguaje.

parseInt(cadena,base): Recibe una cadena y una base (opcional). Devuelve un valor numérico resultante de convertir la cadena en un número en la base indicada. Ejemplo: parseInt("0xF", 16) devuelve un 15. Si no se le pasa una base, transforma una cadena en un número.

parseFloat(cadena): Convierte la cadena en un número y lo devuelve.

```
// A number
parseInt('42px'); // 42 (parseInt lee hasta donde puede)
parseFloat('3.14 €'); // 3.14

// A string
String(42); // '42'
(42).toString(); // '42'

// A boolean
Boolean(0); // false
Boolean(''); // false
Boolean('0'); // true (una cadena no vacía siempre es truthy)
```

Valores *truthy* y *falsy*
Cuando JavaScript necesita interpretar un valor como booleano (por ejemplo, en la condición de un if), aplica una conversión implícita a true o false. Solo existen ocho valores *falsy*; todo lo demás es *truthy*.

Valores falsy (se convierten a false)	Ejemplos de valores truthy
false, 0, -0, 0n, '', null, undefined, NaN	'0', 'false',' ', [], {}, cualquier número distinto de 0

Truco para recordarlo: **falsy = inexistente, cero, falso o error numérico**.

```
const carrito = [];

if (carrito) {
  console.log('Este bloque SÍ se ejecuta'); // un array vacío es truthy
}

if (carrito.length) {
  console.log('Este bloque NO se ejecuta'); // carrito.length es 0, que es falsy
}
```

Nota: Este último ejemplo es un patrón muy habitual en desarrollo profesional: en lugar de escribir if (carrito.length > 0), muchos equipos escriben directamente if (carrito.length), apoyándose en que 0 es *falsy*. Es válido, pero conviene usarlo con cuidado: si carrito.length pudiera no existir (por ejemplo, si carrito fuera undefined), el código lanzaría un error antes de llegar a evaluar la condición.

```
const nombre = '';

if (nombre) {
  console.log('Hay nombre');
} else {
  console.log('No hay nombre');
}
```

1.12 Operadores

Operadores aritméticos

Operador	Descripción	Ejemplo
+	Suma	5 + 2 = 7
-	Resta	5 – 2 = 3
*	Multiplicación	5 * 2 = 10
/	División	5 / 2 = 2.5
%	Resto(modulo)	5 % 2 = 1
**	Potencia	5 ** 2 = 25
++ / --	Incremento / decrement	let x = 5; x++; = 6

```
// El operador % es muy habitual para comprobar paridad
```

```
function esPar(numero) {
  return numero % 2 === 0;
}
console.log(esPar(4)); // true
console.log(esPar(7)); // false
```

Operadores de asignación

Operador	Equivalente a
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y
x ??= y	x = x ?? y (asigna solo si x es null o undefined=

Operadores de incremento y decremento

Operador	Equivalente a
var1++	Var1 = var1 + 1
var1--	Var1 = var1 - 1

Operadores de comparación

Operador	Equivalente a
=== / !===	Igual / distinto (sin conversión de tipo; siempre recomendados)
== / !=	Igual / distinto (con conversión de tipo; evitar)
> / <	Mayor que / menor que
>= / <=	Mayor o igual que / menor o igual que

Operadores lógicos

Operador	Descripción	Ejemplo
&&	Y lógico	edad >=18 && tieneDni
||	O lógico	esAdmin || esEditor
!	Negación	!estaActivo
??	Coalescencia nula (usa el valor de la derecha solo si el de la izquierda es null o undefined)	Nombre ?? 'Anónimo'
?.	Encadenamiento opcional (accede a una propiedad solo si el objeto existe)	usuario?.direccion?.ciudad

```
const descuentoAplicado = 0;

// || trata 0 como falsy, así que aquí NO se comporta como se espera
const descuentoMostrado1 = descuentoAplicado || 'Sin descuento';
console.log(descuentoMostrado1); // 'Sin descuento' (incorrecto: 0 SÍ es un descuento válido)
```

```
// ?? solo entra en juego si el valor es null o undefined
const descuentoMostrado2 = descuentoAplicado ?? 'Sin descuento';
console.log(descuentoMostrado2); // 0 (correcto: respeta el valor 0)
```

Importante: Este es uno de los errores más frecuentes en código profesional: usar || para poner un valor por defecto cuando en realidad se necesita ??. || sustituye cualquier valor *falsy* (0, '', false), mientras que ?? solo sustituye null o undefined. Si un 0, una cadena vacía o un false son valores legítimos en tu aplicación, usa siempre ??.

1.13 Comentarios al código

Los comentarios son texto que el motor de JavaScript ignora al ejecutar el programa, pero que resulta fundamental para que otras personas (o el propio autor, meses después) entiendan el código.

Comentarios de una línea

```
// Calcula el precio final aplicando el IVA vigente
const precioConIva = precioBase * 1.21;
```

Comentarios multilínea

```
/*
  Este bloque calcula el precio final de un pedido,
  incluyendo IVA y gastos de envío según la zona.
*/
const precioFinal = calcularPrecioFinal(pedido);
```

Comentarios de documentación (JSDoc)

En proyectos profesionales, las funciones se documentan con un formato estandarizado llamado **JSDoc**, que además permite que el editor de código (VS Code) muestre autocompletado y ayuda contextual al usar esa función desde cualquier otro fichero. Una vez creada la función, escribe encima /** y pulsa INTRO, VSCode te generará parte del código, solo tienes que rellenar el tipo y la descripción.

```
/**
 * Calcula el precio final de un producto aplicando el IVA.
 * @param {number} precioBase - Precio sin impuestos.
 * @param {number} [tipoIva=0.21] - Tipo de IVA a aplicar (por defecto, 21%).
 * @returns {number} Precio final con IVA incluido.
 */
function calcularPrecioConIva(precioBase, tipoIva = 0.21) {
  return Number((precioBase * (1 + tipoIva)).toFixed(2));
}
```

Importante: Un comentario que explica qué hace una línea obvia (// suma uno a contador sobre contador++;) no aporta valor y ensucia el código. Un buen comentario explica el por qué de una decisión que no es evidente con solo leer el código (por

ejemplo, por qué se aplica un descuento especial en una condición concreta). El mejor comentario, en muchos casos, es un nombre de variable o de función bien elegido que hace innecesario el propio comentario.

1.14 Depurando

La mejor forma de depurar es ejecutando el código paso a paso. Esa funcionalidad nos la dan los navegadores (al menos casi todos).

Al pulsar F12 en el navegador se nos abre un conjunto de herramientas, entre ellas un depurador. Otra forma de abrir estas herramientas es pulsando con el botón derecho en el navegador y elegir la opción "Inspeccionar".

Actividades - Operadores, Prompt, …

Actividad 1. Si nos fijamos en estas líneas de código

```
num1 = 5;
resultado = num1 = 3; // ¿Cuánto vale aquí 'resultado'?
resultado = num1 == 3; // ¿Cuánto vale aquí 'resultado'?
```

Nota: Vamos a aprovechar esta actividad para aprender a utilizar el depurador del navegador y 'observar' una variable.

Actividad 2. Escribir el código necesario para que realice esta operación "v1 * v2 + v3" y muestre el resultado por consola. Siendo los valores de v1, v2 y v3:

```
v1=2  v2=4  v3=5
```

A continuación, hacer lo mismo, pero con esta operación "v1 + v2 * v3". Antes de ejecutarlo, pensad qué resultado obtendremos. ¿Has acertado?

¿Qué orden sigue JS para realizar las operaciones? ¿Cómo podemos cambiarlo?

Actividad 3. ¿Cuál sería el resultado de las siguientes operaciones? a=5; a/=2; a+=1; a*=3; a--; a++;

Actividad 4. Suma tres números tecleados por usuario y calcula la media, mostrando en la consola:

```
"La suma es: RESULTADO_SUMA"
"La media es: MEDIA"
```

Actividad 5. Pide al usuario una cantidad de "millas náuticas" y muestra la equivalencia en metros.

Actividad 6. Dado el siguiente código:

```
let a = 5;
let b = a++;
```

Y este otro:

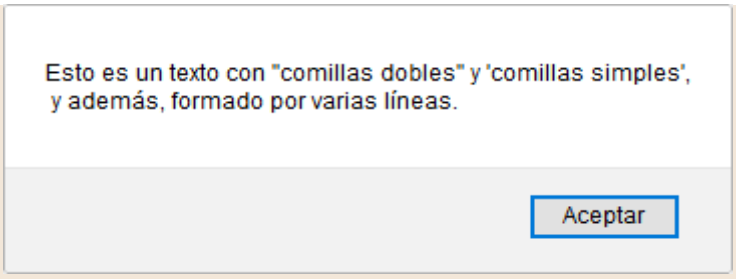
```
let a = 5;
let b = ++a;
```

¿Tendrá la variable 'b' el mismo valor en ambos scripts? ¿Por qué?

Actividad 7. ¿Que mostrará el siguiente código en la consola? ¿Por qué?

```
let foo = 'hola';
console.log( parseInt(foo) );
```

Actividad 8. Mostrar un mensaje con alert, como el que vemos en la imagen:



Actividad 9. Escribe una función generarCodigoDescuento() que devuelva una cadena con el formato DESC- seguido de un número entero aleatorio de 4 cifras (entre 1000 y 9999).

2. Estructuras de control

Hasta ahora el código se ha ejecutado siempre de arriba abajo, línea a línea. Las estructuras de control permiten decidir qué código se ejecuta (condicionales) y repetir código varias veces (bucles).

2.1 Sentencias condicionales: if, else if, else

Una sentencia condicional ejecuta un bloque de código solo si se cumple una condición. La condición es cualquier expresión que JavaScript pueda interpretar como truthy o falsy.

if simple

```
const totalCarrito = 45;


if (totalCarrito > 50) {
  console.log('Envío gratuito');
}
```

if / else

Añade un bloque alternativo que se ejecuta cuando la condición no se cumple.

```
const totalCarrito = 45;


if (totalCarrito > 50) {
  console.log('Envío gratuito');
} else {
  console.log('Gastos de envío: 4.95 €');
}
```

if / else if / else

Permite encadenar varias condiciones que se evalúan en orden. En cuanto una se cumple, se ejecuta su bloque y se ignoran las siguientes.

```
function calcularGastosEnvio(totalCarrito) {
  if (totalCarrito >= 50) {
    return 0;
```

```
  } else if (totalCarrito >= 30) {
    return 2.95;
  } else {
    return 4.95;
  }
}

console.log(calcularGastosEnvio(60)); // 0
console.log(calcularGastosEnvio(35)); // 2.95
console.log(calcularGastosEnvio(10)); // 4.95
```

Importante: Escribe siempre las llaves { }, incluso cuando el bloque tenga una sola sentencia. JavaScript permite omitirlas para bloques de una línea, pero esto es una fuente clásica de errores: si en el futuro se añade una segunda línea al bloque sin añadir las llaves, esa segunda línea queda fuera del if sin que salte ningún error, y el programa falla de forma silenciosa. Los linters profesionales suelen obligar a poner siempre las llaves.

if anidados

Un if puede contener otro if dentro de su bloque, para comprobar condiciones adicionales solo cuando la primera se cumple.

```
function puedeComprarAlcohol(edad, tieneIdentificacion) {
  if (edad >= 18) {
    if (tieneIdentificacion) {
      return 'Venta permitida';
    } else {
      return 'Debe presentar identificación';
    }
  }

  return 'Venta no permitida';
}
```

Nota: El ejemplo anterior también podría resolverse con una sola condición usando el operador &&: if (edad >= 18 && tieneIdentificacion). Anidar es útil cuando cada nivel necesita su propia rama else con un mensaje distinto, como en este caso; si solo hay un resultado final, combinar condiciones con operadores lógicos suele ser más legible que anidar.

El operador ternario: una alternativa compacta al if / else

Cuando un if / else solo sirve para decidir el valor que se asigna a una variable, existe una forma más compacta de escribirlo: el **operador ternario**. Se llama así porque es el único operador de JavaScript que trabaja con tres operandos: una condición, un valor si es verdadera y un valor si es falsa.

```
// Con if / else: cuatro líneas para asignar un único valor
let tipoEntrada;
if (edad >= 18) {
  tipoEntrada = 'Adulto';
} else {
  tipoEntrada = 'Menor';
}

// Con operador ternario: la misma decisión, en una línea
const edad = 16;
const tipoEntradaTernario = edad >= 18 ? 'Adulto' : 'Menor';
console.log(tipoEntradaTernario); // 'Menor'
```

La sintaxis es condición ? valorSiVerdadero : valorSiFalso. Al ser una expresión (devuelve un valor, no solo ejecuta código), se puede usar directamente dentro de otra expresión, como una plantilla literal:

```
const stock = 0;
console.log(`Estado: ${stock > 0 ? 'Disponible' : 'Agotado'}`);
// Estado: Agotado
```

Nota: El operador ternario es adecuado para decisiones simples de una sola línea que producen un valor. Si la lógica requiere ejecutar varias sentencias, o hay más de dos caminos posibles, un if / else if / else normal es más legible. Encadenar varios ternarios seguidos (a ? b : c ? d : e) es una práctica que la mayoría de guías de estilo profesionales desaconsejan por su baja legibilidad.

2.2 La sentencia switch

Cuando una misma variable se compara contra muchos valores posibles, encadenar else if se vuelve repetitivo. La sentencia switch resuelve ese caso concreto: compara una expresión contra una lista de valores (case) y ejecuta el bloque correspondiente al primero que coincida.

```
function descripcionEstadoPedido(estado) {
  switch (estado) {
    case 'pendiente':
      return 'El pedido está pendiente de confirmación';
    case 'enviado':
      return 'El pedido ya ha salido del almacén';
    case 'entregado':
      return 'El pedido ha sido entregado';
    default:
      return 'Estado desconocido';
  }
```

```
}
console.log(descripcionEstadoPedido('enviado')); // 'El pedido ya ha salido del almacén'
```

default es opcional, y se ejecuta cuando ningún case coincide con el valor evaluado. Es equivalente al else final de una cadena de **if / else if**.

También se puede utilizar el switch utilizando un rango de valores en el "case" en vez de un único valor:

```
let num = 5
switch (true) {
    case num > 0:
        ...
        break;
    case (num >= 5 && num < 10):
        ...
        break;
    default:
…
        break;
}
```

El problema del fall-through

Cuando un bloque case no termina con return o break, la ejecución continúa cayendo al siguiente case, aunque su valor ya no coincida con la expresión evaluada. Este comportamiento se llama fall-through y es una de las fuentes de errores más comunes con switch.

```
function descripcionEstadoConError(estado) {
  let mensaje;

  switch (estado) {
    case 'pendiente':
      mensaje = 'Pendiente'; // falta el break
    case 'enviado':
      mensaje = 'Enviado';
      break;
    default:
      mensaje = 'Desconocido';
  }

  return mensaje;
}
```

```
console.log(descripcionEstadoConError('pendiente')); // 'Enviado' (¡no era lo esperado!)
```

2.3 Bucles: while y do...while

Un bucle repite un bloque de código mientras se cumpla una condición. while y do...while son los dos bucles más básicos: la diferencia está en cuándo se comprueba la condición.

while
Comprueba la condición **antes** de cada repetición. Si la condición es falsa desde el principio, el bloque no se ejecuta ni una sola vez.

```
let entradasDisponibles = 3;

while (entradasDisponibles > 0) {
  console.log(`Quedan ${entradasDisponibles} entradas`);
  entradasDisponibles--;
}

console.log('Entradas agotadas');
```

Importante: Toda variable que aparezca en la condición del bucle debe modificarse dentro de su bloque (en el ejemplo, entradasDisponibles--). Olvidar ese paso es la causa más frecuente de un bucle infinito: la condición nunca deja de cumplirse y el programa se queda bloqueado (en un navegador, la pestaña deja de responder).

do...while
Comprueba la condición **después** de cada repetición, lo que garantiza que el bloque se ejecute siempre al menos una vez, incluso si la condición ya es falsa desde el principio. Es útil, por ejemplo, para pedir un dato al usuario hasta que sea válido.

```
let edadIntroducida;

do {
  edadIntroducida = parseInt(prompt('Introduce tu edad (0-120):'));
} while (edadIntroducida < 0 || edadIntroducida > 120)

console.log(`Edad registrada: ${edadIntroducida}`);
```

Nota: Recuerda que prompt siempre devuelve un string (o null si se cancela), por lo que hay que convertirlo con parseInt antes de compararlo numéricamente, como se explicó en el apartado de conversión de tipos.

2.4 El bucle for

Cuando se conoce de antemano cuántas veces hay que repetir un bloque, for es la opción más habitual: reúne en una sola línea la inicialización, la condición y la actualización del contador, en lugar de repartirlas como en un while.

```
for (let i = 1; i <= 5; i++) {
  console.log(`Línea de factura número ${i}`);
}
```

Las tres partes del paréntesis se ejecutan en un orden concreto:

Parte	Cuando se ejecuta	En el ejemplo
Inicialización	Una sola vez, al empezar	let i = 1
Condición	Antes de cada vuelta	i <= 5
Actualización	Después de cada vuelta	i++

También puede recorrer en sentido descendente, o avanzar de dos en dos, cambiando la actualización:

```
// Cuenta atrás
for (let i = 5; i >= 1; i--) {
  console.log(i);
}

// Solo números pares
for (let i = 0; i <= 10; i += 2) {
  console.log(i);
}
```

Nota: Como norma general: usa for cuando el número de repeticiones se conoce de antemano (recorrer un rango, repetir algo N veces), y while cuando depende de una condición externa que no se puede calcular por adelantado (esperar a que el usuario introduzca un dato válido, repetir hasta que ocurra algo). Más adelante, se verán además for...of y métodos de array como alternativa específica para recorrer colecciones de datos.

2.5 for...in y for...of

Los dos recorren "algo", pero recorren cosas distintas y están pensados para casos distintos. Aquí tienes ambos explicados uno junto al otro.

for...of: recorre valores de un iterable

Recorre directamente los valores de una colección iterable: arrays, strings, y otras estructuras que se verán más adelante (como los Map y Set, o las colecciones de nodos del DOM).

```
const productos = ['Teclado', 'Ratón', 'Monitor'];
```

```
for (const producto of productos) {
  console.log(producto);
}
// 'Teclado'
// 'Ratón'
// 'Monitor'
```

for...in: recorre las claves de un objeto
for...in recorre las propiedades **enumerables** de un objeto, dándote su nombre (la clave), no su valor directamente.

```
const producto = { nombre: 'Teclado', precio: 45, disponible: true };

for (const clave in producto) {
  console.log(`${clave}: ${producto[clave]}`);
}
// 'nombre: Teclado'
// 'precio: 45'
// 'disponible: true'
```

Fíjate en que dentro del bucle hace falta producto[clave] (notación de corchetes) para obtener el valor; for...in solo te da el nombre de la propiedad.

2.6 Sentencias de ruptura: break y continue

Dentro de un bucle, a veces hace falta salir antes de que la condición se cumpla de forma natural, o saltarse una vuelta concreta sin terminar el bucle entero. Para eso existen break y continue.

break: termina el bucle por completo
Sale inmediatamente del bucle, sin ejecutar el resto de sus vueltas.

```
// Busca el primer número, hasta 100, divisible entre 7 y entre 4 a la vez
let numeroEncontrado = null;

for (let i = 1; i <= 100; i++) {
  if (i % 7 === 0 && i % 4 === 0) {
    numeroEncontrado = i;
    break; // ya no hace falta seguir comprobando el resto
  }
}

console.log(numeroEncontrado); // 28
```

continue: se salta la vuelta actual

Interrumpe únicamente la iteración actual y pasa directamente a comprobar la condición de la siguiente, sin ejecutar el resto del bloque en esa vuelta.

```
// Suma solo los números impares entre 1 y 10
let sumaImpares = 0;

for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue; // se salta los números pares
  }
  sumaImpares += i;
}

console.log(sumaImpares); // 25
```

Nota: break ya se había usado antes, dentro de un switch, con la misma idea: interrumpir la ejecución en ese punto. En un bucle, además, evita seguir comprobando condiciones que ya no aportan nada, lo cual mejora el rendimiento cuando el bucle recorre muchos elementos.

2.7 Control de errores: try, catch, finally y throw

Hasta ahora, cuando algo fallaba (acceder a una propiedad de null provoca un TypeError, usar una variable que no existe provoca un ReferenceError...), el programa se detenía en esa línea y el resto del código no llegaba a ejecutarse. Para depurar en la consola es útil, pero en una página real no es aceptable que un dato inesperado deje toda la aplicación rota. La sentencia try...catch permite intentar ejecutar un bloque de código y, si se produce un error, reaccionar a él en lugar de que el programa se detenga.

Estructura básica
Consta de tres bloques, de los cuales el tercero es opcional:

```
try {
  // código que puede fallar
} catch (error) {
  // qué hacer si ha fallado
} finally {
  // se ejecuta siempre (opcional)
}
```

try: contiene el código que se quiere vigilar. Se ejecuta línea a línea; en cuanto una línea produce un error, se abandona el bloque (las líneas restantes no se ejecutan) y el control pasa a catch.
catch (error): solo se ejecuta si ha habido un error. La variable error (puede llamarse como se quiera) es un objeto con, al menos, dos propiedades: name (el tipo de error, por ejemplo TypeError) y message (la descripción).

finally: se ejecuta siempre, haya habido error o no. Es el lugar para las acciones que deben ocurrir en cualquier caso (cerrar un mensaje de carga, reactivar un botón...).
Un primer ejemplo con un error real del motor de JavaScript:

```
const usuario = null;


try {
  console.log(usuario.nombre); // TypeError: usuario es null
  console.log('Esta línea no llega a ejecutarse');
} catch (error) {
  console.log(error.name + ': ' + error.message);
}


console.log('El programa continúa');
```

Salida por consola:

```
TypeError: Cannot read properties of null (reading 'nombre')
El programa continúa
```

La segunda línea del try se salta, el catch muestra el error y, lo más importante, el programa sigue ejecutándose después del bloque. Sin el try...catch, la ejecución se habría detenido en la línea del TypeError.

Lanzar errores propios: throw
No solo el motor de JavaScript produce errores: el propio código puede lanzar uno cuando detecta una situación que no puede continuar, con la sentencia throw seguida de un objeto de error. Los más habituales son Error (genérico), TypeError (el tipo de dato no es el esperado) y RangeError (un valor fuera del rango permitido). Al ejecutarse un throw la función se interrumpe, igual que con return, y el error se propaga hacia fuera hasta que encuentra un catch que lo recoja.

Ejemplo de uso: una función que calcula el precio final de un producto tras aplicar un descuento y que se niega a trabajar con datos inválidos, y un programa que revisa una lista de pedidos sin que un pedido erróneo detenga a los demás:

```
function calcularPrecioFinal(precio, descuento) {
  if (typeof precio !== 'number' || typeof descuento !== 'number') {
    throw new TypeError('El precio y el descuento deben ser números');
  }
  if (precio < 0) {
    throw new RangeError('El precio no puede ser negativo');
  }
  if (descuento < 0 || descuento > 100) {
    throw new RangeError('El descuento debe estar entre 0 y 100');
  }
  return precio - precio * descuento / 100;
```

```
}


const pedidos = [
  { producto: 'Teclado', precio: 45, descuento: 20 },
  { producto: 'Ratón', precio: -5, descuento: 10 },
  { producto: 'Monitor', precio: 189, descuento: 150 },
  { producto: 'Webcam', precio: '60', descuento: 10 }
];

for (const pedido of pedidos) {
  try {
    const precioFinal = calcularPrecioFinal(pedido.precio, pedido.descuento);
    console.log(pedido.producto + ': ' + precioFinal.toFixed(2) + ' €');
  } catch (error) {
    console.log(pedido.producto + ': rechazado (' + error.message + ')');
  } finally {
    console.log('Revisado: ' + pedido.producto);
  }
}
```

Salida por consola:

```
Teclado: 36.00 €
Revisado: Teclado
Ratón: rechazado (El precio no puede ser negativo)
Revisado: Ratón
Monitor: rechazado (El descuento debe estar entre 0 y 100)
Revisado: Monitor
Webcam: rechazado (El precio y el descuento deben ser números)
Revisado: Webcam
```

Con el primer pedido todo va bien: el try termina sin errores, el catch se salta y se ejecuta el finally. En los otros tres, la función lanza un error con throw, el try se abandona justo en esa llamada (el console.log de la línea siguiente no se ejecuta), el catch muestra el mensaje y el bucle pasa al siguiente pedido. Observa que el finally se ejecuta en los cuatro casos.
La ventaja de este diseño es el reparto de responsabilidades: la función solo sabe validar y avisar de que algo va mal (throw), y quien la llama decide qué hacer con el problema (mostrar un mensaje, pedir el dato otra vez, registrar el fallo...).

Nota: finally se ejecuta incluso si dentro del try (o del catch) hay un return: la función no termina hasta que se ha ejecutado el finally. Por eso es el sitio adecuado para las tareas de limpieza.

Importante: try...catch no sustituye a un if. Para situaciones previsibles y habituales (por ejemplo, comprobar si un campo está vacío) lo correcto es un if. Los errores con throw se reservan para situaciones en las que el código no puede continuar con normalidad. Además, solo captura los errores que se producen mientras se ejecuta su bloque: un error de sintaxis impide que el script llegue a arrancar, y los errores de código asíncrono (el que se ejecuta más tarde, como una petición de red) ocurren cuando el try ya ha terminado, por lo que requieren otras técnicas que se verán más adelante, al estudiar las peticiones asíncronas.

Actividades – Estructuras de control

De aquí en adelante, cosas que hay que tener en cuenta a la hora de realizar los ejercicios:
- Cuidado con lo que devuelve el prompt => es un string
- Si no funciona el código => mirar la consola (F12) para ver si hay errores de sintaxis.
- Si no funciona el código y no hay errores de sintaxis => depurar (F12)

Actividad 1. Crea un programa que pida al usuario un número y diga si es par.

Actividad 2. Crea un programa que pida al usuario dos números y diga cuál es el mayor de ellos.

Actividad 3. Crea un programa que pida al usuario dos números y diga si el primero es múltiplo del segundo.

Actividad 4. Crea un programa que pida al usuario dos números. Si el segundo no es cero, mostrará el resultado de dividir el primero entre el segundo. Por el contrario, si el segundo número es cero, escribirá "Error: No se puede dividir entre cero".

Actividad 5. Crea un programa que pida al usuario dos números y diga si son iguales o, en caso contrario, cuál es el mayor de ellos.

Actividad 6. Necesitamos un programa que muestre los números del 10 al 20, ambos inclusive. Y a continuación, muestre los números del 20 al 10, es decir, de mayor a menor.

Actividad 7. Crea un programa que escriba en pantalla los números del 1 al 50 que sean múltiplos de 3.

Actividad 8. Crea un programa que muestre los números del 100 al 200 (ambos incluidos) que sean divisibles entre 7 y a la vez entre 3.

Actividad 9. Crea un programa que muestre la tabla de multiplicar del 9:

```
9 x 0 = 0
9 x 1 = 9
9 x 2 = 18
…
9 x 10 = 90
```

Actividad 10. Crea un programa que muestre los primeros ocho números pares: 2 4 6 8 10 12 14 16.

Actividad 11. Crea un programa que muestre los números del 15 al 5, descendiendo.

Actividad 12. Crea un programa que pida al usuario el ancho (por ejemplo, 4) y el alto (por ejemplo, 3) y muestre un rectángulo formado por esa cantidad de asteriscos:

```
****
****
****
```

Actividad 13. Lo mismo que el ejercicio anterior, pero esta vez mostrará una diagonal con otro carácter:

O***
*O**
**O*
***O

Actividad 14. Crea un programa que pida al usuario su identificador y contraseña de manera indefinida hasta que introduzca el identificador “alibaba” y la contraseña “sesamo”. Si al tercer intento no ha introducido los datos correctos, mostrará un mensaje diciendo que el identificador/contraseña no son correctos.
Actividad 15. Crea un "calculador de cuadrados": pedirá al usuario un número y mostrará su cuadrado. Se repetirá mientras el número introducido no sea cero.
Actividad 16. Crea un programa que contenga un bucle sin fin que muestra el mensaje "Hola" con alert. Cuando llegue a 10 vueltas, se detendrá.

3. Arrays

Un array agrupa varios valores en una sola variable, en un orden determinado. Aquí se estudian a fondo, junto con los métodos que los convierten en una de las herramientas más usadas de todo el lenguaje.

3.1 Crear arrays y acceder a sus elementos

Para crear un array, podemos utilizar varias sintaxis distintas:

```
let array_name = [item1, item2, ...];
let array_name = new Array(item1, item2, ...);
let array_name = Array(item1, item2, ...);
```

Podemos crear un array en JS con elementos de distinto tipo. Por ejemplo, en JS, puedo hacer:

```
let miArray = [“Juan”, 4, “Badajoz”];
```

Lo más profesional es crear un array con corchetes, separando sus elementos por comas. Cada elemento tiene una posición numérica, llamada índice, que empieza siempre en 0.

```
const productos = ['Teclado', 'Ratón', 'Monitor'];

console.log(productos[0]); // 'Teclado'
console.log(productos[2]); // 'Monitor'
console.log(productos[5]); // undefined: no existe esa posición
console.log(productos.length); // 3
```

Para acceder al último elemento sin conocer de antemano la longitud del array, el método moderno .at() admite índices negativos, contando desde el final:

```
console.log(productos.at(-1)); // 'Monitor' (el último)
console.log(productos[productos.length - 1]); // 'Monitor' (la forma tradicional, equivalente)
```

Nota: Un array puede contener valores de distintos tipos a la vez ([1, 'dos', true] es válido), pero en código profesional se procura que todos los elementos de un mismo array sean del mismo tipo o tengan la misma forma, precisamente para poder recorrerlos y procesarlos con los métodos que se ven en este tema sin tener que comprobar el tipo de cada elemento uno por uno.

Importante: Evita crear arrays con new Array(). new Array(3) no crea un array con un elemento de valor 3, sino un array vacío de longitud 3 (tres huecos sin definir), un comportamiento que sorprende a quien no lo conoce. La sintaxis de corchetes ([]), usada en todos los ejemplos de este tema, no tiene ninguno de estos casos especiales y es la forma recomendada en cualquier situación.

3.2 Añadir, eliminar y modificar elementos

Método	Qué hace
.push(valor)	Añade un elemento al final; devuelve la nueva longitud
.pop()	Elimina y devuelve el último elemento
.unshift(valor)	Añade un elemento al principio; devuelve la nueva longitud
.shift()	Elimina y devuelve el primer elemento

pop(), elimina el último elemento del array y lo devuelve. El array original se modifica y su longitud disminuye en 1 elemento.

```
let array = [1, 2, 3];
let ultimo = array.pop();
// ahora array = [1, 2], ultimo = 3
```

push(), añade un elemento al final del array. El array original se modifica y aumenta su longitud en 1 elemento. (También es posible añadir más de un elemento a la vez)

```
let array = [1, 2, 3];
array.push(4);
// ahora array = [1, 2, 3, 4]
```

shift(), elimina el primer elemento del array y lo devuelve. El array original se ve modificado y su longitud disminuida en 1 elemento.

```
let array = [1, 2, 3];
let primero = array.shift();
// ahora array = [2, 3], primero = 1
```

unshift(), añade un elemento al principio del array. El array original se modifica y aumenta su longitud en 1 elemento. (También es posible añadir más de un elemento a la vez)

```
let array = [1, 2, 3];
array.unshift(0);
// ahora array = [0, 1, 2, 3]
```

Ejemplo completo:

```
const carrito = ['Teclado'];

carrito.push('Ratón');
console.log(carrito); // ['Teclado', 'Ratón']

carrito.unshift('Monitor');
console.log(carrito); // ['Monitor', 'Teclado', 'Ratón']

const ultimoEliminado = carrito.pop();
console.log(ultimoEliminado); // 'Ratón'
console.log(carrito); // ['Monitor', 'Teclado']
```

Modificar un elemento por su índice

```
const productos = ['Teclado', 'Ratón', 'Monitor'];
productos[1] = 'Ratón inalámbrico';
console.log(productos); // ['Teclado', 'Ratón inalámbrico', 'Monitor']
```

Nota: Esto no contradice lo visto sobre const: la variable productos sigue apuntando al mismo array en todo momento, solo se modifica su contenido interno, igual que ocurría al hacer usuario.rol = 'editor' con un objeto declarado con const.

3.3 Recorrer un array: for, for...of y forEach

for clásico

Usando el índice, exactamente como se vio anteriormente, pero con la condición ligada a .length.

```
const productos = ['Teclado', 'Ratón', 'Monitor'];

for (let i = 0; i < productos.length; i++) {
  console.log(`${i}: ${productos[i]}`);
}
```

for...in

Si se quiere recorrer un array y queremos saber el índice de sus elementos, usar for…in:

```
for(let indice in miArray)
{
    console.log("El elemento " + indice + " toma el valor" + miArray[indice]);
}
```

for...of

Recorre directamente los **valores** de un array, sin necesidad de manejar un índice.

```
for (const producto of productos) {
  console.log(producto);
```

```
}
// 'Teclado'
// 'Ratón'
// 'Monitor'
```

Al no depender de un índice, for...of es más legible cuando no hace falta saber en qué posición está cada elemento. Como es una variante de for, admite break y continue con normalidad.

forEach(): recorrer con un método del propio array
forEach() ejecuta una función (un *callback*, es decir, una función que se pasa como argumento a otra) una vez por cada elemento del array.

```
productos.forEach((producto, indice) => {
  console.log(`${indice}: ${producto}`);
});
```

El *callback* recibe hasta tres argumentos: el elemento actual, su índice, y el array completo (los dos últimos son opcionales, y se pueden omitir si no se necesitan).

Importante: forEach() siempre devuelve undefined, y no admite break ni continue: una vez empieza, recorre el array entero sin posibilidad de interrumpirlo antes de tiempo. Si en algún momento hace falta salir del bucle antes de llegar al final, hay que usar for o for...of en su lugar.

Ejercicios · Arrays
Ejercicio 1. Crea una función que recorra una cadena y separe los caracteres con un guión (Que no aparezca guión al final).
Ejercicio 2. Función que defina una cadena, la corte y la meta en un array. Luego debe recorrer el array e indicar que en el índice X del array está tal cadena.

Array multidimensionales (no es más arrays dentro de arrays):

```
let miArrayFila1 = ["fila1 col1", "fila1 col2"];        // Array unidimensional
let miArrayFila2 = ["fila2 col1", "fila2 col2"];        // Array unidimensional
let miArrayTabla = [miArrayFila1, miArrayFila2];    // Array de dos dimensiones
```

Para acceder a un elemento de un array multidimensional:

```
miArrayTabla[1][0] => estaría accediendo a la columna 1 de la fila 2
```

3.4 Métodos de arrays

slice, corta un rango de elementos del array y lo devuelve. Tiene 2 parámetros, la posición inicial y la posición final (no incluida). Si no se incluye la posición inicial, corta hasta el final del array. No modifica el array original.

```
const numbers = [1, 2, 3, 4, 5]
```

```
console.log(numbers.slice()) // -> copia todos los elementos
console.log(numbers.slice(0)) // -> copia todos los elementos
console.log(numbers.slice(1, 4)) // -> [2,3,4] // No incluye el índice 4
```

splice, es el método más versátil (y el que más confusión genera) para modificar un array en cualquier punto, no solo en los extremos. Su forma general es array.splice(inicio, cantidadAEliminar, ...elementosAInsertar), y **modifica el array original**. Devuelve un array con los elementos eliminados.

```
const numbers = [1, 2, 3, 4, 5]
console.log(numbers.splice()) // -> elimina todos los ítems

const numbers = [1, 2, 3, 4, 5]
console.log(numbers.splice(0, 1)) // -> elimina el primer elemento

const numbers = [1, 2, 3, 4, 5, 6]
console.log(numbers.splice(3, 3, 7, 8, 9)) // -> [1, 2, 3, 7, 8, 9] //Elimina los últimos 3 elementos y los reemplaza por 7,8,9.

const pedidos = ['P001', 'P002', 'P003', 'P004'];

// Eliminar: desde el índice 1, elimina 2 elementos
const eliminados = pedidos.splice(1, 2);
console.log(eliminados); // ['P002', 'P003'] (lo eliminado)
console.log(pedidos); // ['P001', 'P004']

// Insertar: desde el índice 1, elimina 0 elementos, e inserta dos nuevos
pedidos.splice(1, 0, 'P005', 'P006');
console.log(pedidos); // ['P001', 'P005', 'P006', 'P004']

// Reemplazar: desde el índice 0, elimina 1 elemento, e inserta uno nuevo en su lugar
pedidos.splice(0, 1, 'P001-B');
console.log(pedidos); // ['P001-B', 'P005', 'P006', 'P004']
```

Importante: Nunca uses el operador delete para quitar un elemento de un array (delete pedidos[1]): deja un hueco (el elemento pasa a ser undefined) sin acortar el array ni reordenar el resto de índices, lo que casi nunca es el resultado que se busca. Para eliminar de verdad un elemento, usa siempre splice() (si conoces su posición) o los métodos de transformación que veremos más adelante, como filter().

Crear un array de tamaño fijo: Array(n) y .fill()
Es una forma rápida de generar un array del tamaño que quieras, todo con el mismo valor inicial.

```
const ochoValoresX = Array(8).fill('X');
console.log(ochoValoresX); // ['X','X','X','X','X','X','X','X']
```

.concat()
Se emplea para concatenar los elementos de varios arrays.

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];
const array3 = array1.concat(array2); // [1, 2, 3, 4, 5, 6]
```

[...array1, ...array2] hace lo mismo y es la forma preferida en código moderno. .concat() sigue siendo válido y lo verás en código existente.

.join(separador)
Operación inversa de .split(): convierte un array en una cadena de texto. Sin argumento, usa una coma por defecto.

```
const names = ['Ana', 'Luis', 'Marta'];

console.log(names.join());      // 'Ana,Luis,Marta'
console.log(names.join(''));    // 'AnaLuisMarta'
console.log(names.join(' '));   // 'Ana Luis Marta'
console.log(names.join(', '));  // 'Ana, Luis, Marta'
```

Array.isArray()
Comprueba si un valor es realmente un array. Útil porque, como vimos, typeof [] devuelve 'object', no 'array'; esta es la forma correcta de distinguirlo.

```
console.log(Array.isArray([1, 2, 3])); // true
console.log(Array.isArray('hola'));    // false
```

.indexOf() / .lastIndexOf()
.indexOf() busca la **primera** coincidencia; .lastIndexOf(), la **última**. Ambas devuelven -1 si no encuentran nada, y funcionan igual con un carácter suelto o con una palabra completa. Igual que con cadenas.

```
const numbers = [1, 2, 3, 4, 5]
console.log(numbers.indexOf(5)) // -> 4
console.log(numbers.indexOf(0)) // -> -1

const numbers = [1, 2, 3, 4, 5, 3, 1, 2]
console.log(numbers.lastIndexOf(2)) // 7
console.log(numbers.lastIndexOf(0)) // -1
```

.toString()
Convierte un array a texto, uniendo sus elementos con comas (equivale a .join() sin argumentos).

```
const numeros = [1, 2, 3];
```

```
console.log(numeros.toString()); // '1,2,3'
```

Ejercicios · Arrays
Ejercicio 3. Crea un array de elementos y muestra dichos elementos por pantalla separados por "#", ahora haz lo mismo con uno bidimensional:
let anidado1 = ["anidado1", "anidado2", "anidado3"]
let anidado2 = ["a1", "a2", "a3"]
let matriz = [anidado1, anidado2]
Recorre la matriz y muestra los elementos separados por "#", mostrando el mensaje: "Indice X: elem1#elem2#elem3"
Ejercicio 4. Mostrar los elementos de un array del final al principio usando pop.
Ejercicio 5. Nos dan un array de frutas ["banana", "naranja", "mango", "limon"]. Crear un script que pregunte al usuario qué fruta quiere buscar en el array, si no existe dicha fruta mostrará "Esta fruta no existe en el array" en caso contrario mostrará "Sí hay aguacate en el array". La fruta se puede poner en mayúsculas y minúsculas y la seguirá encontrando.

reverse(), modifica un array colocando sus elementos en el orden inverso a su posición original:

```
let array = [1, 2, 3];

array.reverse();

// ahora array = [3, 2, 1]
```

sort(), ordena los elementos de un array en orden ascendente.

```
let fruits = ["Banana", "Orange", "Apple", "Mango"];

fruits.sort();  //  fruits = ["Apple", "Banana", "Mango", "Orange"];
```

Ejercicios · Arrays
Ejercicio 6. Si tenemos una cadena en minúsculas "miel". ¿Cómo podemos transformarla en una cadena con la primera letra en mayúsculas(Miel)?
Ejercicio 7. Nos dan la lista de la compra en un array(["Leche", "Café", "Té", "Miel"). Crea un botón para añadir "Carne" al principio de la lista de la compra y "Azúcar" al final de la lista. Si ya se ha añadido "Carne" o "Azúcar" no se vuelve a añadir, se muestra el mensaje "Ya se ha añadido este producto a la lista de la compra".
Crea un botón "Alergias" que pida un producto al que eres alérgico y lo elimina de la lista de la compra, indicando "He eliminado X producto de la cesta porque eres alérgico", si no se encuentra el producto en la cesta, mostrará "No existen alergias en la cesta".
Crea un botón "Modificar" que pida un producto a buscar y si lo encuentra modifique su valor, por ejemplo, que busque "Té" y te permita modificarlo a "Té verde".

3.5 Buscar y comprobar elementos

Método	Qué devuelve
.includes(valor)	true/false según si el array contiene ese valor exacto
.indexOf(valor)	El índice de la primera coincidencia, o -1 si no está
.find(callback)	El primer elemento que cumple la condición, o undefined
.findIndex(callback)	El índice del primer elemento que cumple la condición, o -1
.some(callback)	true si al menos un elemento cumple la condición
.every(callback)	true solo si todos los elementos cumplen la condición

.includes() e .indexOf() sirven para valores simples y exactos. Cuando los elementos son objetos, hace falta una condición más elaborada, y para eso están find, findIndex, some y every, que reciben un callback igual que forEach.

Método "includes"
Se utiliza para verificar si un array o una cadena de texto contiene un elemento específico o un valor determinado. Esta función retorna true si el elemento está presente y false si no lo está. Ejemplo: Ver si el array tiene el elemento 3.

```
let array = [1, 2, 3, 4, 5];
let elemento = 3;
let estaPresente = array.includes(elemento);

if (estaPresente) {
  alert(`El elemento ${elemento} está presente en el array.`);
} else {
  alert(`El elemento ${elemento} no está presente en el array.`);
}
```

Método "find"
El método .find() es una herramienta moderna (introducida en ES6) que permite buscar el primer elemento de un array que cumpla una condición específica. A diferencia de indexOf() que busca coincidencias exactas, .find() puede buscar basándose en criterios complejos mediante una función anónima.
Sintaxis básica:

```
array.find(function(elemento, índice, array) {
   // Retorna true si el elemento cumple la condición
   // Retorna false si no la cumple
   });
```

Características importantes
- Retorna el primer elemento que cumple la condición.
- Si ningún elemento cumple la condición, retorna undefined.

- Deja de buscar en cuanto encuentra el primer elemento que cumple la condición (es eficiente).
- No modifica el array original.

Ejemplos:

Ejemplo 1: Búsqueda básica con números

```
// Encontrar el primer número mayor que 10
  const numeros = [3, 8, 12, 5, 15, 20];
  const primerMayorQue10 = numeros.find(function (numero) {
    return numero > 10;
  });
```

Versión con función flecha (más compacta):

```
  const numeros = [3, 8, 12, 5, 15, 20];
  const primerMayorQue10 = numeros.find(num => num > 10);
  console.log(primerMayorQue10); // 12
```

Ejemplo 2: Búsqueda en array de objetos (caso real)

Este es el uso más potente de .find():

```
// Array de estudiantes
  const estudiantes = [
    { dni: "12345678A", nombre: "Ana", edad: 20, nota: 7.5 },
    { dni: "87654321B", nombre: "Carlos", edad: 22, nota: 8.2 },
    { dni: "11223344C", nombre: "María", edad: 21, nota: 9.1 }
  ];

  // Buscar estudiante por DNI
  const estudianteBuscado = estudiantes.find(function (estudiante) {
    return estudiante.dni === "87654321B";
  });

  console.log(estudianteBuscado);
  // { dni: "87654321B", nombre: "Carlos", edad: 22, nota: 8.2 }
  console.log(estudianteBuscado.nombre); // "Carlos"
```

Ejemplo 3: Diferencia entre .find() y .indexOf()

```
  const frutas = ["manzana", "naranja", "plátano", "manzana"];

  // indexOf() busca coincidencias EXACTAS y devuelve el ÍNDICE
  const indice = frutas.indexOf("naranja");
  console.log(indice); // 1 (posición del array)
```

```
// find() busca con condiciones y devuelve el ELEMENTO
const frutaEncontrada = frutas.find(fruta => fruta === "naranja");
console.log(frutaEncontrada); // "naranja" (el elemento en sí)

// find() puede buscar con condiciones complejas
const frutaLarga = frutas.find(fruta => fruta.length > 6);
console.log(frutaLarga); // "manzana" (primera con más de 6 letras)
```

Ejemplo comparativo:

```
const productos = [
    { id: 1, nombre: "Portátil", precio: 800 },
    { id: 2, nombre: "Ratón", precio: 25 },
    { id: 3, nombre: "Teclado", precio: 60 }
];

// find() - Solo el PRIMERO que cumple
const primerBarato = productos.find(p => p.precio < 100);
console.log(primerBarato); // { id: 2, nombre: "Ratón", precio: 25 }

// filter() - TODOS los que cumplen
const todosBaratos = productos.filter(p => p.precio < 100);
console.log(todosBaratos); // Array con Ratón y Teclado
```

Más ejemplos:

```
const inventario = [
  { sku: 'TC-01', nombre: 'Teclado', stock: 12 },
  { sku: 'RT-02', nombre: 'Ratón', stock: 0 },
  { sku: 'MN-03', nombre: 'Monitor', stock: 5 }
];

const raton = inventario.find(articulo => articulo.sku === 'RT-02');
console.log(raton); // { sku: 'RT-02', nombre: 'Ratón', stock: 0 }

const hayAgotados = inventario.some(articulo => articulo.stock === 0);
console.log(hayAgotados); // true

const todoDisponible = inventario.every(articulo => articulo.stock > 0);
console.log(todoDisponible); // false
```

Nota: La sintaxis { clave: valor } usada en inventario es un objeto literal, ya adelantado de forma puntual en temas anteriores. Su sintaxis completa, junto con clases y el resto de programación orientada a objetos, se estudia más adelante.

Ejercicios · Arrays

Ejercicio 8. Crea un programa que pida al usuario 6 números y luego los muestre en orden inverso (pista: usa un array para almacenarlos y "for" para mostrarlos).

Ejercicio 9. Crea un programa que pregunte al usuario cuantos números va a introducir (por ejemplo, 5), le pida todos esos números, los guarde en un array y finalmente calcule y muestre la media de esos números.

Ejercicio 10. Crea un array de 100 elementos. El valor del primer elemento será 1, valor del segundo elemento será 2, y así sucesivamente, de tal forma que el último elemento del array tendrá el valor 100. Calcula la media de todos los elementos del array.

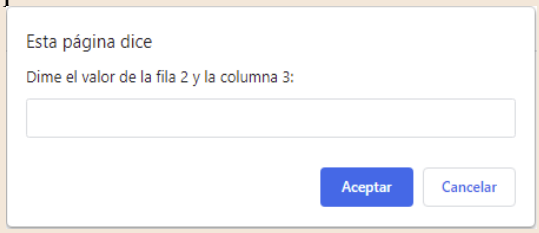
Ejercicio 11. Un programa que pida al usuario 10 números, calcule su media y luego muestre los que están por encima de la media.

Ejercicio 12. Desarrolla un programa que pedirá nombres al usuario hasta que se introduzca un nombre vacío, momento en el que dejarán de pedirse más nombres y se mostrará en pantalla la lista de los nombres que se han introducido ordenados alfabéticamente.

Ejercicio 13. Escribe un script para generar un array de dos dimensiones (matriz). Se le pedirá al usuario primero el tamaño de la matriz (que será cuadrada):



Y a continuación le irá pidiendo el valor de cada celda.



Por último, mostrar en un alert mostrando en cada celda el valor correspondiente a su posición en la matriz.

Ejercicio 14. Crear un programa que pida una nota (número) y muestre la calificación según la nota. Si la nota no es entre 0 y 10 mostrará el error "Nota errónea":

0-3: Muy deficiente
3-5: Insuficiente
5-6: Suficiente
6-7: Bien
7-9: Notable
9-10: Sobresaliente

Ejercicio 15. Programa que pida 2 números y muestre todos los números impares que hay entre estos 2 números introducidos (ambos inclusive) separados por un guión. Sin usar arrays y sin que quede un guión al final.

Ejercicio 16. Realiza un script que pida cadenas de texto hasta que se pulse cancelar en un confirm. Al salir con "cancelar" deben mostrarse todas las cadenas concatenadas con un guión.

Ejercicio 17. Realiza un script que escriba una pirámide del 1 al 30 de la siguiente forma:

1
22
333
4444
55555
666666

Ejercicio 18. Realiza un script que escriba los números del 1 al 500, que indique cuales son múltiplos de 4 o de 9 que cada 5 líneas impresas muestre una línea horizontal.
Ejemplo:
4 es múltiplo de 4
8 es múltiplo de 4
9 es múltiplo de 9
12 es múltiplo de 4
16 es múltiplo de 4
_____-
18 es múltiplo de 9
20 es múltiplo de 4
24 es múltiplo de 4
27 es múltiplo de 9
28 es múltiplo de 4

Ejercicio 19. Realiza un script que pida número de filas y columnas y escriba una tabla. Dentro cada una de las celdas deberá escribirse un número consecutivo en orden descendente. Si, por ejemplo, la tabla es de 7×5 los números irán del 35 al 1.
Ejercicio 20. Que pida un número y genere la secuencia de Fibonnaci entre 0 y dicho número (ambos inclusive). Los números deben mostrarse con un espacio entre ellos. La secuencia se calcula sumando los 2 anteriores, siendo la secuencia así: 0,1,1,2,3,5,8,13...
Ejercicio 21. Hacer una utilidad para sumar metros en una app de running. Al pulsar el botón + se suman metros 1 a 1 y al pulsar el botón - se restan (usa alert para mostrar el resultado y un prompt o variable para definir los metros). El botón "+ Decenas" suma los metros en decenas (de 10 en 10), es decir que, si estoy en 22 metros, al darle al botón iré a 30 metros y si vuelvo a pulsar voy a 40 metros. El botón "- Decenas" hace lo contrario, si estoy en 23 metros, iré a 20 metros y si vuelvo a pulsar iré a 10 metros y así hasta que sea cero, no se debe permitir poner metros en negativo.

4. Funciones

Las funciones permiten agrupar código para reutilizarlo con distintos datos de entrada, en lugar de repetirlo. Este tema cubre las distintas formas de declararlas, cómo reciben y devuelven datos, y un concepto clave para entender cómo funciona JavaScript por dentro: las clausuras.

4.1 Declaración de funciones: declaration vs.expression

Existen dos formas principales de definir funciones, con una diferencia de comportamiento importante entre ellas.

Function declaration

Se define con la palabra clave function seguida de un nombre, como sentencia independiente.

```
function calcularIva(precio) {
  return precio * 0.21;
}
```

Function expression
Se define asignando una función, sin nombre propio, a una variable.

```
const calcularIva = function (precio) {
  return precio * 0.21;
};
```

Importante: La diferencia práctica más relevante está en el hoisting. Una function declaration se eleva por completo, con su cuerpo incluido, así que se puede llamar antes de la línea donde aparece escrita en el código. Una function expression se comporta como cualquier variable const o let: existe, pero no se puede usar antes de la línea donde se le asigna la función (queda en la temporal dead zone).

```
console.log(calcularIvaDeclarada(100)); // funciona: 21

function calcularIvaDeclarada(precio) {
  return precio * 0.21;
}

console.log(calcularIvaExpresion(100)); // ReferenceError: Cannot access before initialization

const calcularIvaExpresion = function (precio) {
  return precio * 0.21;
};
```

4.2 Parámetros y argumentos

Parámetro es el nombre que recibe un dato dentro de la definición de la función; argumento es el valor concreto que se pasa al llamarla. En function calcularIva(precio), precio es el parámetro; en calcularIva(100), 100 es el argumento.

Parámetros por defecto
Permiten asignar un valor que se usa automáticamente cuando no se pasa ese argumento al llamar a la función.

```
function calcularPrecioConIva(precioBase, tipoIva = 0.21) {
  return Number((precioBase * (1 + tipoIva)).toFixed(2));
}

console.log(calcularPrecioConIva(100)); // 121 (usa el 0.21 por defecto)
console.log(calcularPrecioConIva(100, 0.10)); // 110 (usa el valor indicado)
```

Parámetros *rest*: agrupar argumentos sueltos

Cuando una función debe aceptar un número indeterminado de argumentos, el operador ... delante del último parámetro los agrupa a todos en un único array.

```
function sumarTodos(...numeros) {
  let total = 0;

  for (const numero of numeros) {
    total += numero;
  }

  return total;
}

console.log(sumarTodos(1, 2, 3)); // 6
console.log(sumarTodos(10, 20, 30, 40)); // 100
```

Nota: El array es el tipo de dato pensado precisamente para agrupar varios valores en uno solo; se estudiará más adelante, junto con sus métodos. Por ahora, basta con saber que ...numeros convierte "todos los argumentos que sobren" en una lista que se puede recorrer con for...of.

Paso de argumentos: valor frente a referencia

Ya se explicó que los primitivos se copian por valor y los objetos y arrays se copian por referencia. Esa misma regla se aplica al pasar argumentos a una función.

```
function intentarDuplicar(numero) {
  numero = numero * 2;
}

let cantidad = 5;
intentarDuplicar(cantidad);
console.log(cantidad); // 5 (no cambia: se pasó una COPIA del valor)

function aplicarDescuento(producto) {
  producto.precio = producto.precio * 0.9; // modifica una propiedad del objeto
}

const articulo = { nombre: 'Ratón', precio: 20 };
aplicarDescuento(articulo);
console.log(articulo.precio); // 18 (SÍ cambia: se pasó una referencia al mismo objeto)
```

Importante: Modificar un objeto recibido como parámetro dentro de una función afecta al objeto original, fuera de la función, porque ambas variables (la del código que llama y el parámetro dentro de la función) apuntan al mismo objeto en memoria. Esto es

intencionado y muy útil, pero también una fuente de errores si no se tiene en cuenta: una función que "solo debía leer" un objeto puede acabar modificándolo sin que quien la usa lo espere.

4.3 El valor de retorno

return termina la ejecución de la función de inmediato y devuelve el valor indicado a quien la llamó. Si una función no tiene ningún return, o tiene un return sin valor, devuelve undefined.

```
function registrarVisita() {
  console.log('Visita registrada');
  // no hay return: la función devuelve undefined
}

const resultado = registrarVisita(); // imprime 'Visita registrada'
console.log(resultado); // undefined
```

Retorno anticipado (*early return*)
Un patrón muy habitual en código profesional es salir de la función en cuanto se detecta un caso no válido, en lugar de anidar toda la lógica dentro de un único if. Esto se conoce como *guard clause* (cláusula de guarda).

```
// Con anidamiento: la lógica principal queda "escondida" dentro del if
function procesarPedidoAnidado(pedido) {
  if (pedido.total > 0) {
    if (pedido.stockDisponible) {
      return 'Pedido confirmado';
    } else {
      return 'Sin stock';
    }
  } else {
    return 'Pedido vacío';
  }
}

// Con guard clauses: cada caso no válido sale de inmediato
function procesarPedido(pedido) {
  if (pedido.total <= 0) {
    return 'Pedido vacío';
  }

  if (!pedido.stockDisponible) {
    return 'Sin stock';
```

```
  }

  return 'Pedido confirmado';
}
```

Nota: Las dos funciones anteriores hacen exactamente lo mismo. La segunda versión suele considerarse más legible en equipos profesionales: cada condición de error se resuelve y se aparta de inmediato, dejando el caso "todo va bien" al final, sin anidamiento.

4.4 Arrow functions

Las arrow functions (funciones flecha) son una sintaxis más compacta para escribir function expressions, introducida en ES6. Se usan de forma constante en JavaScript moderno.

```
// Function expression tradicional
const calcularIva = function (precio) {
  return precio * 0.21;
};

// La misma función, como arrow function
const calcularIvaFlecha = (precio) => {
  return precio * 0.21;
};

// Si el cuerpo es una única expresión, se puede omitir "return" y las llaves
const calcularIvaCorta = (precio) => precio * 0.21;
```

Situación	**Sintaxis**
Un solo parámetro	precio => precio * 0.21 (los paréntesis son opcionales, pero muchas guías de estilo profesionales piden ponerlos siempre: (precio) => ...)
Varios parámetros	(a, b) => a + b (paréntesis obligatorios)
Sin parámetros	() => console.log('hola')
Cuerpo de una sola expresión	Sin llaves ni return: el valor de la expresión se devuelve automáticamente (*implicit return*)
Cuerpo con varias sentencias	Llaves obligatorias, y return explícito si se quiere devolver algo

Nota: En la práctica profesional, las arrow functions se usan sobre todo para funciones cortas y para pasarlas como argumento a otras funciones (algo muy habitual con los métodos de array, y con la gestión de eventos). Las function declarations siguen siendo una opción perfectamente válida, especialmente para funciones con nombre propio que se llaman por sí solas, no como argumento de otra función.

Ejercicios · Repaso

Ejercicio 1. Escribe una función clasificarImc(imc) que reciba un índice de masa corporal (número) y devuelva, usando if / else if / else, uno de estos textos: "Bajo peso" (menos de 18.5), "Normal" (entre 18.5 y 24.9), "Sobrepeso" (entre 25 y 29.9) u "Obesidad" (30 o más).

Ejercicio 2. Reescribe la siguiente función usando el operador ternario, de forma que ocupe una sola línea dentro del cuerpo de la función:

```
function obtenerEtiquetaStock(unidades) {
    if (unidades > 0) {
        return 'En stock';
    } else {
        return 'Agotado';
    }
}
```

Ejercicio 3. Escribe una función nombreDiaSemana(numeroDia) que reciba un número del 1 al 7 y devuelva, usando switch, el nombre del día correspondiente (1 = "Lunes" ... 7 = "Domingo"). Si el número no está entre 1 y 7, debe devolver "Día no válido".

Ejercicio 4. Explica, sin ejecutarlo, qué imprimirá este código y por qué (presta atención a los break):

```
function categoria(puntos) {
  let resultado;
  switch (true) {
    case puntos >= 100:
      resultado = 'Oro';
      break;
    case puntos >= 50:
      resultado = 'Plata';
    case puntos >= 0:
      resultado = 'Bronce';
      break;
    default:
      resultado = 'Sin categoría';
  }
  return resultado;
}
console.log(categoria(60));
```

Ejercicio 5. Escribe un bucle for que recorra los números del 1 al 30 y muestre por consola "Fizz" si el número es múltiplo de 3, "Buzz" si es múltiplo de 5, "FizzBuzz" si es múltiplo de ambos, y el propio número en cualquier otro caso.

Ejercicio 6. Identifica el error en el siguiente código y corrígelo:

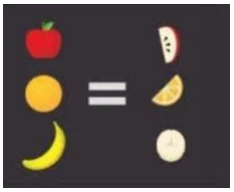
```
let intentos = 0;

while (intentos < 3) {
  console.log('Intento fallido');
```

}
Ejercicio 7. Usando for y continue, escribe un bucle que recorra los números del 1 al 20 y sume solo aquellos que sean múltiplos de 3, mostrando el resultado final por consola.

4.5 Transformar arrays: map, filter y reduce

Estos tres métodos son, probablemente, los más usados de todo JavaScript moderno. Los tres reciben un callback, y ninguno modifica el array original: todos devuelven un resultado nuevo.



map(): transforma cada elemento
Devuelve un array nuevo, de la misma longitud que el original, con el resultado de aplicar el callback a cada elemento.
Su sintaxis básica es:

```
let nuevoArray = arrayOriginal.map((elemento, índice, array) => {
  // Código para transformar el elemento
  return nuevoElemento;
});
```

Donde elemento es el elemento actual que se está transformando.
- Indice(opcional): es el índice del elemento actual en el array.
- Array(opcional): es el array original en el que se está realizando la transformación.
La función toma un elemento del array original como parámetro de entrada y debe retornar el nuevo valor que se asignará al elemento correspondiente en el nuevo array resultante. El nuevo array tendrá la misma longitud que el array original.

```
const precios = [19.99, 45.5, 12.0];
const preciosConIva = precios.map(precio => Number((precio * 1.21).toFixed(2)));

console.log(preciosConIva); // [24.19, 55.06, 14.52]
console.log(precios); // [19.99, 45.5, 12] (sin modificar)
```



filter(): selecciona elementos
Devuelve un array nuevo, con solo los elementos para los que el *callback* devuelve true. Puede tener una longitud menor (o igual) que el original.
Su sintaxis es:

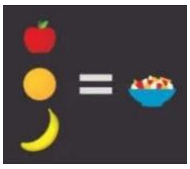
```
let nuevoArray = arrayOriginal.filter(función(elemento, índice, array) => { … });
```

- Función: es la función que se llama una vez por cada elemento del array original. Esta función debe retornar true para incluir el elemento en el nuevo array o false para omitirlo. Los elementos que cumplan con la condición especificada por la función se copiarán al nuevo array.
- Elemento: es el elemento actual que se está evaluando.
- *Indice*(opcional): es el índice del elemento actual en el array.
- *Array*(opcional): es el array original en el que se está realizando el filtrado.

```
const inventario = [
 { nombre: 'Teclado', stock: 12 },
 { nombre: 'Ratón', stock: 0 },
```

```
  { nombre: 'Monitor', stock: 5 }
];

const disponibles = inventario.filter(articulo => articulo.stock > 0);
console.log(disponibles); // [{ nombre: 'Teclado', ... }, { nombre: 'Monitor', ... }]
```



reduce(): acumula un único resultado

reduce() es el más flexible, y también el que más cuesta interiorizar al principio. Recorre el array acumulando un resultado en una variable (el *acumulador*), que va pasando de una iteración a la siguiente.

La sintaxis básica es:

```
const resultado = array.reduce((acumulador, elemento, índice, array) => {
  // Código para acumular el valor
  return nuevoValorDelAcumulador;
}, valorInicial);
```

- Acumulador: es el valor acumulado que se va actualizando en cada iteración.
- Elemento: elemento actual de la iteración.
- Indice(opcional): es el índice del elemento actual en el array.
- Array(opcional): es el array original en el que se está realizando la reducción.
- Valor inicial: Un valor inicial para el acumulador, si se omite, la primera iteración comenzará con el elemento del array.

La función que se le pasa por parámetro debe retornar el nuevo valor del acumulador en cada iteración. El resultado final será el valor acumulado después de procesar todos los elementos del array.

```
const precios = [19.99, 45.5, 12.0];

const total = precios.reduce((acumulador, precioActual) => {
  return acumulador + precioActual;
}, 0); // el 0 final es el valor inicial del acumulador

console.log(total); // 77.49
```

Es útil seguir la ejecución paso a paso la primera vez que se usa:

Iteración	acumulador (antes)	precioActual	acumulador (después)
1	0 (valor inicial)	19.99	19.99
2	19.99	45.5	65.49
3	65.49	12.0	77.49

Importante: El segundo argumento de reduce() (el valor inicial del acumulador) casi nunca debería omitirse. Si se omite, reduce() toma el primer elemento del array como valor inicial y empieza a iterar desde el segundo, lo cual puede dar resultados inesperados (o un error, si el array está vacío). Indicar siempre el valor inicial explícitamente hace el comportamiento predecible en cualquier caso.

Encadenar los tres métodos

Como map() y filter() devuelven un array nuevo, se pueden encadenar entre sí y terminar con un reduce(), formando una pequeña "cadena de montaje" de datos.

```
const inventario = [
  { nombre: 'Teclado', precio: 45, stock: 12 },
  { nombre: 'Ratón', precio: 20, stock: 0 },
  { nombre: 'Monitor', precio: 150, stock: 5 }
];

const valorTotalDisponible = inventario
  .filter(articulo => articulo.stock > 0)
  .map(articulo => articulo.precio * articulo.stock)
  .reduce((total, valorArticulo) => total + valorArticulo, 0);

console.log(valorTotalDisponible); // 45*12 + 150*5 = 1290
```

Nota: Este estilo (encadenar filter, map y reduce en lugar de escribir un único bucle for con varias condiciones dentro) se conoce como programación funcional, y es el estándar en JavaScript profesional moderno: cada método hace una sola cosa, el código se lee casi como una frase ("filtra los disponibles, calcula su valor, súmalo todo"), y ninguno modifica los datos originales.

Ejercicios · Map, Filter, Reduce

Ejercicio 1. Obtener de un array solo las cadenas que tengan una longitud mayor a 5 caracteres. palabras = ['manzana', 'pera', 'banana', 'uva', 'sandía', 'kiwi'];

Ejercicio 2. Filtrar un array de elementos y mostrar solo los que estén en las posiciones pares (en los índices pares del array). Usad este array: numeros = [10, 20, 30, 40, 50, 60, 70, 80, 90];

Ejercicio 3. Encuentra el número mayor del array: numeros = [12, 45, 67, 89, 23, 56, 34, 78]

Ejercicio 4. Sumarle el IVA a todos los elementos del array.
const precios = [100, 200, 300, 400, 500];

Ejercicio 5. Array de nombres y ponerlos todos en mayúsculas.
const nombres = ['Juan', 'Pedro', 'Luis', 'Ana', 'Maria'];

Ejercicio 6. Dado un array de objetos que representan productos, crea un array con solo los nombres de los productos.
const productos = [
{ id: 1, nombre: "Laptop", precio: 1200 },
{ id: 2, nombre: "Teclado", precio: 50 },
{ id: 3, nombre: "Ratón", precio: 25 },

```
];
```

4.6 Ordenar y combinar arrays

sort(): cuidado con el orden por defecto

sort() ordena el array modificándolo directamente (a diferencia de map, filter y reduce). Por defecto, convierte todos los elementos a texto y los ordena alfabéticamente, incluso si son números.

```
const cantidades = [10, 1, 21, 2];


console.log(cantidades.sort()); // [1, 10, 2, 21] (¡orden alfabético, no numérico!)
```

Importante: Para ordenar números correctamente, hay que pasarle a sort() una función de comparación: recibe dos elementos (a y b) y debe devolver un número negativo si a va antes que b, positivo si va después, o 0 si son iguales. Restar ambos valores resuelve el caso numérico de forma directa.

```
const cantidades = [10, 1, 21, 2];


console.log(cantidades.sort((a, b) => a - b)); // [1, 2, 10, 21] (ascendente)
console.log(cantidades.sort((a, b) => b - a)); // [21, 10, 2, 1] (descendente)
```

Combinar arrays: concat() y el operador *spread*

El operador ..., usado para *agrupar* varios argumentos en un array (parámetros *rest*), tiene el efecto contrario cuando se usa dentro de un array literal: expande un array existente en sus elementos individuales.

```
const categoriaOficina = ['Teclado', 'Ratón'];
const categoriaPantallas = ['Monitor', 'Webcam'];


// Con concat()
const catalogoCompleto1 = categoriaOficina.concat(categoriaPantallas);


// Con el operador spread (forma preferida en código moderno)
const catalogoCompleto2 = [...categoriaOficina, ...categoriaPantallas];


console.log(catalogoCompleto2); // ['Teclado', 'Ratón', 'Monitor', 'Webcam']
```

El mismo operador permite crear una copia independiente de un array, sin compartir referencia con el original.

```
const original = [1, 2, 3];
const copia = [...original];


copia.push(4);
```

```
console.log(original); // [1, 2, 3] (no afectado)
console.log(copia); // [1, 2, 3, 4]
```

Nota: Esto último es importante: const copia = original; (sin ...) NO crea una copia, sino una segunda variable apuntando al mismo array, exactamente igual que ocurría con los objetos. El operador spread es una de las formas más habituales de crear una copia real en JavaScript moderno.

4.7 Desestructuración de arrays

La desestructuración permite extraer varios elementos de un array y asignarlos a variables individuales, en una sola línea, en lugar de acceder a cada posición por separado.

```
const coordenadas = [40.4168, -3.7038];

// Sin desestructurar
const latitud = coordenadas[0];
const longitud = coordenadas[1];

// Con desestructuración: mismo resultado, una sola línea
const [lat, lon] = coordenadas;
console.log(lat, lon); // 40.4168 -3.7038
```

Se pueden omitir posiciones que no interesen, dejando una coma vacía, y asignar valores por defecto para posiciones que podrían no existir:

```
const fila = ['Ana', 28, 'Madrid'];
const [nombre, , ciudad] = fila; // se salta la edad, en el índice 1
console.log(nombre, ciudad); // 'Ana' 'Madrid'

const [primero, segundo = 'Sin definir'] = ['único valor'];
console.log(segundo); // 'Sin definir' (no había un segundo elemento)
```

Combinada con el operador rest, permite separar los primeros elementos del resto:

```
const clasificacion = ['Oro', 'Plata', 'Bronce', 'Cuarto', 'Quinto'];
const [primerPuesto, segundoPuesto, ...resto] = clasificacion;

console.log(primerPuesto); // 'Oro'
console.log(resto); // ['Bronce', 'Cuarto', 'Quinto']
```

5. El objeto Date

Date representa un instante concreto en el tiempo. A diferencia de Math, sí es un constructor: cada fecha es una nueva instancia, creada con new Date().

Crear una fecha

```
const ahora = new Date(); // fecha y hora actuales
const fechaConcreta = new Date(2026, 8, 11); // 11 de septiembre de 2026
const desdeTexto = new Date('2026-09-11'); // a partir de una cadena en formato ISO
```

- Si no se le pasa ningún parámetro, coge la fecha actual.
- Si se le pasa 1 parámetro, se indica el número de milisegundos que han transcurrido desde el 1 de Enero de 1970.
- También se le puede pasar un solo parámetro indicando una cadena de texto indicando la fecha (aunque la cadena de texto tiene que tener un formato específico).
- Si se le pasa 3 parámetros: (año, mes, día).
- Si se le pasan 6 parámetros: (año, mes, día, hora, minuto, segundo).

Ejemplos:

```
let fecha = new Date();  //  "La fecha y hora actuales"
let fecha = new Date(10000000000000);  // "Sat Nov 20 2286 18:46:40"
let fecha = new Date(2009, 5, 1);  // 1 de Junio de 2009 (00:00:00)
let fecha = new Date(2009, 5, 1, 19, 29, 39);  // 1 de Junio de 2009 (19:29:39)
```

Importante: El constructor new Date(año, mes, día) numera los meses empezando en 0: enero es 0 y diciembre es 11. En el ejemplo anterior, 8 corresponde a septiembre, no a agosto. Este es uno de los errores más habituales al trabajar con fechas en JavaScript; usar directamente una cadena en formato ISO ('2026-09-11'), como en la tercera línea, evita este problema por completo.

Obtener componentes de una fecha

Método	Qué devuelve
.getFullYear()	Año (4 dígitos)
.getMonth()	Mes, de 0 (enero) a 11 (diciembre)
.getDate()	Día del mes, de 1 a 31
.getDay()	Día de la semana, de 0 (domingo) a 6 (sábado)
.getHours() / .getMinutes()	Hora y minutos

Mostrar una fecha con formato

Para mostrar una fecha al usuario, en vez de construir el texto a mano concatenando los componentes anteriores, se usa toLocaleDateString(), que aplica el formato propio de un idioma o región.

```
const fechaPedido = new Date('2026-09-11');


console.log(fechaPedido.toLocaleDateString('es-ES')); // '11/9/2026'
```

```
console.log(fechaPedido.toLocaleDateString('es-ES', {
  day: 'numeric',
  month: 'long',
  year: 'numeric'
})); // '11 de septiembre de 2026'
```

Operar con fechas

Para sumar o restar días, se combina un método *getter* con su *setter* equivalente:

```
const fechaPedido = new Date('2026-09-11');
const fechaEntregaEstimada = new Date(fechaPedido);
fechaEntregaEstimada.setDate(fechaPedido.getDate() + 5);

console.log(fechaEntregaEstimada.toLocaleDateString('es-ES')); // '16/9/2026'
```

Hemos utilizado setDate, pero hay los mismos métodos "set" para todos los que tengan "get": setMonth(), setDay(), setFullYear()…

Para comparar dos fechas, se convierten a su valor numérico interno (milisegundos desde el 1 de enero de 1970) con getTime(), y se comparan como números:

```
const fechaLimite = new Date('2026-09-15');
const hoy = new Date();

if (hoy.getTime() > fechaLimite.getTime()) {
  console.log('Plazo vencido');
} else {
  console.log('Todavía en plazo');
}
```

Nota: La API nativa de Date es funcional pero verbosa, y algunas operaciones (como calcular la diferencia en días entre dos fechas, o trabajar con zonas horarias) resultan incómodas. En proyectos profesionales es habitual apoyarse en una librería especializada, como *date-fns* o *Day.js*, para ese tipo de cálculos más avanzados.

Ejercicios · Funciones

Ejercicio 1. Escribe una función con, al menos, estos valores: 3, 3.5, "3", "3.5", "7aaa", "aaa7" y "hola" en un array. Recorre el array y muestra por pantalla el resultado de aplicar las funciones parseInt, parseFloat, isNan, Number y String, a cada uno de los elementos del array. ¿Hay algún resultado que no esperabas?

Ejercicio 2. Crea una función llamada "DibujarLinea3", que dibuje una línea con 3 asteriscos en la pantalla.

Ejercicio 3. Crea una función llamada "DibujarCuadrado3x3", que dibuje un cuadrado (en la pantalla) formado por 3 filas con 3 asteriscos cada una (esta función deberá llamar a la función creada en el ejercicio anterior).

Ejercicio 4. Crea una función llamada "DibujarLinea", que dibuje una línea con X asteriscos (en la pantalla). La función tendrá un parámetro de entrada con el número de asteriscos a mostrar.
Ejercicio 5. Crea una función llamada "DibujarCuadrado", que dibuje un cuadrado formado por X filas con X asteriscos cada una (esta función deberá llamar a la función creada en el ejercicio anterior). El script pedirá al usuario el tamaño del cuadrado, y llamará a la función con ese dato.
Ejercicio 6. Crea una función "DibujarRectangulo" que dibuje en pantalla un rectángulo del ancho y alto que se indiquen como parámetros. El script pedirá al usuario el tamaño del rectángulo, y llamará a la función con ese dato.
Ejercicio 7. Crea una función "Cubo" que devuelva el cubo de un número real que se indique como parámetro. La función tendrá que comprobar que el parámetro es un número (o que se pueda convertir a un número), en caso contrario, devolverá false. Prueba esta función para calcular el cubo de 3.2 y el de 5, y también prueba a pasarle "hola" como parámetro. ¿Qué ocurre?
Ejercicio 8. Crea una función "Menor" que devuelva el menor de dos números que recibirá como parámetros.
Ejercicio 9. Crear 4 funciones: PideNumero, EsPositivo, CalculaMitad y HazTodo:
La función PideNumero, pedirá al usuario que introduzca un número y devuelve el número introducido.
Las funciones EsPositivo y CalculaMitad, no imprimen nada en la consola, simplemente devuelven los valores correspondientes.
La función HazTodo no tendrá ningún parámetro, llamará a las otras tres funciones y mostrará la siguiente información en la consola:

'El número X es POSITIVO/NEGATIVO'
'La mitad de X es Y'

Fuera de las funciones, solo habrá una llamada a la función HazTodo.
Ejercicio 10. Escribe una función que dado un string, devuelva un array con las palabras que forman el string. ¿Qué pasa si el string tiene 3 espacios en blanco seguidos? ¿Se puede solucionar?
Ejercicio 11. Escribe una función que se le pase un nombre completo (string) y devuelve el nombre abreviado (es decir, el nombre y la primera letra del primer apellido, seguido de un punto). Escribe también el código necesario para probarla.
Ejercicio 12. Crea una función que oculte parte de una dirección de email. Por ejemplo, si se le pasa "1234567890@gmail.com", devolverá "12345…@gmail.com", es decir, sustituye la 2ª mitad del email (antes de la @) por "...". Escribe también el código necesario para probarla.
Ejercicio 13. Define una función que sustituya los espacios en blanco por un guión y, además, convierta todo el texto en minúsculas. Escribe también el código necesario para probarla.
Ejercicio 14. Escribe una función que reciba dos parámetros, string1 y string2, y que devuelva true si string1 contiene a string2, o false en caso contrario.
Ejercicio 15. Crea una función que genere un número aleatorio, entre dos valores dados. Utiliza esta función para simular el funcionamiento de un dado.
Ejercicio 16. Define una función que devuelva una cadena de texto con el día actual. Además, hay que pasarle el separador. Suponiendo que el separador es "-", el formato de salida será "dd-mm-yyyy".

Ejercicio 17. En una empresa las facturas vencen a los 20 días. Crear una función que tendrá como parámetro una fecha con el formato 'dd-mm-YYYY' y devolverá la fecha de vencimiento (con el mismo formato). Hay que tener en cuenta que, si la fecha de vencimiento cae en fin de semana, habrá que mostrar la fecha del viernes anterior.
Ejercicio 18. Hacer una función que reciba dos parámetros (mes y año). Con esos datos tiene que mostrar la siguiente información (por ejemplo, si le pasamos el mes de Octubre y el año 2020:

Octubre (2020)

Semana 1: 1(Ju) 2(Vi) 3(Sa) 4(**Do**)
Semana 2: 5(Lu) 6(Ma) 7(Mi) 8(Ju) 9(Vi) 10(Sa) 11(**Do**)
Semana 3: 12(Lu) 13(Ma) 14(Mi) 15(Ju) 16(Vi) 17(Sa) 18(**Do**)
Semana 4: 19(Lu) 20(Ma) 21(Mi) 22(Ju) 23(Vi) 24(Sa) 25(**Do**)
Semana 5: 26(Lu) 27(Ma) 28(Mi) 29(Ju) 30(Vi) 31(Sa)

Ejercicio 19. Escribe una función diasHastaFecha(fechaObjetivo) que reciba una fecha en formato 'YYYY-MM-DD' y devuelva el número de días que faltan desde hoy hasta esa fecha (puedes usar getTime() y dividir la diferencia en milisegundos entre 1000 * 60 * 60 * 24).

6. Programación orientada a objetos

6.1 Objetos literales

Un objeto agrupa datos relacionados en pares clave: valor. Ya se ha usado esta sintaxis de forma puntual en temas anteriores, aquí se completa con todo lo que permite un objeto literal.

```
const producto = {
  nombre: 'Teclado mecánico',
  precio: 45,
  disponible: true
};

console.log(producto.nombre); // notación de punto: 'Teclado mecánico'
console.log(producto['precio']); // notación de corchetes: 45
```

Notación de corchetes: claves dinámicas
La notación de punto exige conocer el nombre de la propiedad de antemano, escrito literalmente. La notación de corchetes admite una variable, lo que permite acceder a una propiedad cuyo nombre solo se conoce en tiempo de ejecución.

```
const campoBuscado = 'precio';
console.log(producto[campoBuscado]); // 45
console.log(producto.campoBuscado); // undefined: buscaría una propiedad LITERAL llamada
"campoBuscado"
```

Propiedades y métodos abreviados (*shorthand*)
Cuando el nombre de una variable ya existente coincide con el nombre que se le quiere dar a una propiedad, se puede omitir la repetición:

```
const nombre = 'Ana';
const edad = 28;

// Forma completa
const usuario1 = { nombre: nombre, edad: edad };

// Forma abreviada (property shorthand): mismo resultado
const usuario2 = { nombre, edad };
```

Los objetos también pueden contener funciones como valores de una propiedad, llamadas **métodos**. Igual que con las propiedades, existe una forma abreviada:

```
// Forma completa
const producto1 = {
  nombre: 'Teclado',
  describir: function () {
    return `Producto: ${producto1.nombre}`;
  }
};

// Forma abreviada (method shorthand): mismo resultado
const producto2 = {
  nombre: 'Teclado',
  describir() {
    return `Producto: ${this.nombre}`;
  }
};
```

Nota: En producto2, el método usa this.nombre en lugar de producto2.nombre. Qué es exactamente this y por qué es la forma preferida se explica en el siguiente apartado; de momento basta con saber que, dentro de un método, this permite referirse "al propio objeto", sin tener que repetir su nombre.

Recorrer las propiedades de un objeto

Object.keys(), Object.values() y Object.entries() convierten las propiedades de un objeto en un array.

```
const producto = { nombre: 'Teclado', precio: 45, disponible: true };

console.log(Object.keys(producto)); // ['nombre', 'precio', 'disponible']
console.log(Object.values(producto)); // ['Teclado', 45, true]
console.log(Object.entries(producto)); // [['nombre', 'Teclado'], ['precio', 45], ['disponible', true]]

// Combinado con for...of y desestructuración
```

```
for (const [clave, valor] of Object.entries(producto)) {
  console.log(`${clave}: ${valor}`);
}
```

Eliminar una propiedad y hacer un objeto inmutable

```
delete producto.disponible;
console.log(producto); // { nombre: 'Teclado', precio: 45 }
```

const no impide modificar las propiedades de un objeto. Para conseguir eso, existe Object.freeze():

```
const configuracion = Object.freeze({ tema: 'oscuro' });

configuracion.tema = 'claro'; // no lanza error, pero tampoco hace nada
console.log(configuracion.tema); // 'oscuro' (sin cambios)
```

Comprobar si existe una propiedad

Acceder a una propiedad que no existe no provoca ningún error: el resultado es simplemente undefined. El problema es que ese resultado no permite distinguir entre una propiedad que no existe y una propiedad que existe, pero vale undefined. Para saber con certeza si existe hay dos herramientas: el operador **in**, que comprueba si la propiedad existe en el objeto (o en lo que este hereda), y Object.hasOwn(objeto, clave), que comprueba únicamente las propiedades propias del objeto(sin mirar lo heredado.

```
const producto = { nombre: 'Teclado', precio: 45, descuento: undefined };

console.log(producto.stock);              // undefined (no existe, pero no da error)
console.log(producto.descuento);          // undefined (existe, pero vale undefined)

console.log('precio' in producto);        // true
console.log('stock' in producto);         // false
console.log('descuento' in producto);     // true (existe aunque valga undefined)
console.log(Object.hasOwn(producto, 'stock')); // false
```

Nota: El operador **in** también devuelve true para los métodos que todo objeto hereda de JavaScript, como toString: 'toString' in producto es true, mientras que Object.hasOwn(producto, 'toString') es false. Por eso, para comprobar los datos de tu propio objeto, es preferible Object.hasOwn.

El encadenamiento opcional (?.), resuelve el caso más habitual: leer una propiedad de un objeto anidado que puede no existir, sin que el programa se detenga con un error.

```
const usuario = { nombre: 'Ana' };

console.log(usuario.direccion?.ciudad); // undefined (sin error)
console.log(usuario.direccion.ciudad);  // TypeError: Cannot read properties of undefined
```

Copiar y combinar objetos: el operador spread

Ya se explicó que una variable que contiene un objeto no guarda el objeto, sino una referencia a él, así que const copia = original no crea ninguna copia. El operador spread (...), visto con los arrays, también funciona con objetos: { ...original } crea un objeto nuevo con las mismas propiedades. Si se escriben propiedades después del spread, sustituyen a las que venían de él, lo que permite copiar y modificar en un solo paso. También sirve para combinar varios objetos: si una propiedad aparece en más de uno, gana la última.

```
const original = { nombre: 'Teclado', precio: 45 };

const alias = original;         // NO es una copia: dos nombres para el mismo objeto
alias.precio = 50;
console.log(original.precio);   // 50

const copia = { ...original };  // copia real
copia.precio = 60;
console.log(original.precio);   // 50 (el original no cambia)

// Copiar y modificar a la vez: gana la última propiedad escrita
const rebajado = { ...original, precio: 39.9, oferta: true };
console.log(rebajado);          // { nombre: 'Teclado', precio: 39.9, oferta: true }

// Combinar objetos (valores por defecto + preferencias del usuario)
const porDefecto = { tema: 'claro', idioma: 'es' };
const preferencias = { tema: 'oscuro' };
const ajustes = { ...porDefecto, ...preferencias };
console.log(ajustes);           // { tema: 'oscuro', idioma: 'es' }
```

Importante: El spread hace una copia superficial: copia las propiedades del primer nivel, pero si alguna es a su vez un objeto o un array, la copia y el original siguen compartiendo ese objeto interno. Para una copia completamente independiente (copia profunda) existe structuredClone().

```
const pedido = { cliente: 'Ana', lineas: ['Teclado', 'Ratón'] };

const copiaSuperficial = { ...pedido };
copiaSuperficial.lineas.push('Monitor');
console.log(pedido.lineas);   // ['Teclado', 'Ratón', 'Monitor'] (el array se comparte)

const copiaProfunda = structuredClone(pedido);
copiaProfunda.lineas.push('Webcam');
console.log(pedido.lineas);   // ['Teclado', 'Ratón', 'Monitor'] (sin cambios)
```

Nota: structuredClone() no puede copiar funciones (lanza un DataCloneError) y las instancias de una clase se convierten en objetos normales, sin sus métodos. Antes de que existiera el spread de objetos, la copia superficial se hacía con Object.assign({}, original), que sigue funcionando, pero hoy se escribe casi siempre con spread.

Desestructuración de objetos

La desestructuración de arrays extrae los valores según su posición. En los objetos se extraen según el nombre de la propiedad, por lo que el orden no importa. Las variables se crean con el mismo nombre que la propiedad, escritas entre llaves a la izquierda del igual.

```
const producto = { nombre: 'Teclado', precio: 45, stock: 12 };

// Sin desestructurar
const nombre1 = producto.nombre;
const precio1 = producto.precio;

// Con desestructuración: las variables se llaman igual que las propiedades
const { nombre, precio } = producto;
console.log(nombre, precio); // 'Teclado' 45
```

Igual que con los arrays, se pueden dar valores por defecto (para propiedades que podrían no existir) y recoger el resto de propiedades con rest. Además, si el nombre de la propiedad no es el que se quiere para la variable, se puede renombrar con dos puntos: propiedad: nuevoNombre.

```
// Renombrar (propiedad: nuevoNombre) y valor por defecto (= valor)
const { nombre: titulo, descuento = 0 } = producto;
console.log(titulo);    // 'Teclado'
console.log(descuento); // 0 (la propiedad no existe, se usa el valor por defecto)

// Rest: el resto de propiedades se recogen en un objeto nuevo
const { stock, ...datosPublicos } = producto;
console.log(datosPublicos); // { nombre: 'Teclado', precio: 45 }
```

Donde más se usa es en los parámetros de una función: en lugar de recibir el objeto entero y escribir producto.nombre, producto.precio... dentro de ella, la función declara directamente las propiedades que necesita.

```
function mostrarProducto({ nombre, precio, stock = 0 }) {
  return `${nombre}: ${precio} € (${stock} uds.)`;
}

const teclado = { nombre: 'Teclado', precio: 45, stock: 12 };
const raton = { nombre: 'Ratón', precio: 20 };
```

```
console.log(mostrarProducto(teclado)); // 'Teclado: 45 € (12 uds.)'
console.log(mostrarProducto(raton));   // 'Ratón: 20 € (0 uds.)'
```

Nota: Los dos puntos tienen aquí un significado distinto al de crear un objeto: en const { nombre: titulo } = producto, la desestructuración lee la propiedad nombre y crea la variable titulo, mientras que en const objeto = { nombre: titulo } se crea una propiedad nombre con el valor de la variable titulo.

6.2 this: qué es y cómo se determina

this es una palabra clave especial que, dentro de un método, hace referencia al objeto que ha llamado a ese método. Su valor no depende de dónde se ha escrito la función, sino de cómo se la ha llamado.

```
const producto = {
  nombre: 'Teclado',
  precio: 45,
  aplicarDescuento(porcentaje) {
    return this.precio * (1 - porcentaje);
  }
};

console.log(producto.aplicarDescuento(0.1)); // 40.5
```

Al llamar a producto.aplicarDescuento(0.1), JavaScript establece this como producto, porque es el objeto que aparece a la izquierda del punto en el momento de la llamada. Gracias a eso, el método puede usar this.precio sin necesitar el nombre producto escrito de forma literal, lo que permite reutilizar el mismo método en distintos objetos.

6.3 Clases

Las clases, introducidas en ES6, no son un mecanismo nuevo: son una forma más clara de escribir exactamente el mismo patrón (función constructora más métodos en el prototipo). Por eso se dice que son azúcar sintáctico: hacen el código más legible, sin cambiar cómo funciona por debajo.

```
class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }

  aplicarDescuento(porcentaje) {
    return this.precio * (1 - porcentaje);
  }
}
```

```
const teclado = new Producto('Teclado', 45);
console.log(teclado.aplicarDescuento(0.1)); // 40.5
console.log(teclado instanceof Producto); // true
```

El método constructor se ejecuta automáticamente al crear una instancia con new, y es donde se inicializan sus propiedades. El resto de métodos de la clase se añaden automáticamente al prototipo.

Getters y setters
Permiten definir propiedades que, en apariencia, se leen o se asignan como datos normales, pero que en realidad ejecutan código cada vez que se accede a ellas.

```
class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }

  precioConIva() {
    return Number((this.precio * 1.21).toFixed(2));
  }

  set precio(valorNuevo) {
    if (valorNuevo < 0) {
      console.warn('El precio no puede ser negativo; se ignora el cambio');
      return;
    }
    this._precio = valorNuevo;
  }

  get precio() {
    return this._precio;
  }
}

const teclado = new Producto('Teclado', 45);
console.log(teclado.precioConIva()); // 54.45
teclado.precio = -10; // pasa por el setter, que rechaza el valor
console.log(teclado.precio); // 45 (sin cambios)
```

Nota: En el ejemplo anterior, la propiedad interna se llama _precio (con guion bajo) para no chocar de nombre con el getter/setter público precio. Es una convención muy

extendida para señalar "esto es un detalle interno", aunque el guion bajo, por sí solo, no hace la propiedad realmente privada: sigue siendo accesible desde fuera como teclado._precio.

Campos privados reales: #

Para una privacidad real (no solo una convención de nombre), JavaScript moderno permite declarar campos con el prefijo #, que los hace completamente inaccesibles desde fuera de la clase.

```
class CuentaBancaria {
  #saldo;

  constructor(saldoInicial) {
    this.#saldo = saldoInicial;
  }

  ingresar(cantidad) {
    this.#saldo += cantidad;
  }

  consultarSaldo() {
    return this.#saldo;
  }
}

const cuenta = new CuentaBancaria(100);
cuenta.ingresar(50);
console.log(cuenta.consultarSaldo()); // 150
console.log(cuenta.#saldo); // SyntaxError: propiedad privada inaccesible desde fuera
```

Campos privados con getters y setters

Los campos privados se combinan de forma natural con getters y setters: el valor vive en un campo # que nadie puede tocar desde fuera y la única vía de acceso es el getter y el setter, que pueden validar. Es la versión moderna del ejemplo con _precio, pero con privacidad real.

```
class Producto {
  #precio;

  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;   // pasa por el setter
  }
```

```
  get precio() {
    return this.#precio;
  }

  set precio(valorNuevo) {
    if (valorNuevo < 0) {
      console.warn('El precio no puede ser negativo; se ignora el cambio');
      return;
    }
    this.#precio = valorNuevo;
  }
}

const teclado = new Producto('Teclado', 45);
teclado.precio = -10;          // el setter lo rechaza
console.log(teclado.precio);   // 45
// console.log(teclado.#precio); // SyntaxError: el campo es privado
```

Miembros estáticos

Un método o propiedad static pertenece a la propia clase, no a cada instancia. Se usa para utilidades relacionadas con la clase que no necesitan un objeto concreto para funcionar.

```
class Producto {
  static unidadMonetaria = 'EUR';

  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }
}

console.log(Producto.unidadMonetaria); // 'EUR' (se accede desde la clase, no desde una instancia)
```

Atributos de clase y atributos de instancia

En programación orientada a objetos, un atributo de instancia es un dato que tiene cada objeto con su propio valor (el nombre y el precio de cada producto), y un atributo de clase es un dato único, compartido por todos los objetos de la clase (el IVA, o el número de productos creados hasta ahora). En JavaScript, los primeros son los campos normales y los segundos son los campos static.

Un uso muy habitual de los atributos de clase es llevar la cuenta de los objetos creados, por ejemplo, para asignar identificadores automáticos. Para que nadie pueda manipular ese contador desde fuera, se declara privado además de estático. Un getter también puede ser estático.

```
class Pedido {
  static #ultimoId = 0;   // atributo de clase privado: solo lo ve la propia clase
  id;                     // atributo de instancia: cada pedido tiene el suyo

  constructor() {
    this.id = ++Pedido.#ultimoId;
  }

  static totalCreados() {
    return Pedido.#ultimoId;
  }
}

const pedido1 = new Pedido();
const pedido2 = new Pedido();
console.log(pedido1.id, pedido2.id);  // 1 2
console.log(Pedido.totalCreados());     // 2
```

El método toString()

Cuando JavaScript necesita convertir un objeto en texto (al mostrarlo con alert(), al incluirlo en una plantilla literal, al concatenarlo con un string o al llamar a String(objeto)), invoca automáticamente el método toString() del objeto. Ya has usado toString() con números (42).toString() y con arrays. Todos los objetos lo tienen porque lo heredan de Object, pero la versión por defecto no es nada útil:

```
const sinFormato = { nombre: 'Teclado' };

console.log(`Producto: ${sinFormato}`); // 'Producto: [object Object]'
alert(sinFormato);                      // muestra: [object Object]
```

La solución es definir nuestro propio toString() en la clase (o en el objeto literal). Así se controla qué texto representa a los objetos, y ese texto se usará en todos los sitios donde JavaScript convierte el objeto a String:

```
class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }

  toString() {
    return `${this.nombre} (${this.precio} €)`;
  }
}
```

```
const teclado = new Producto('Teclado', 45);
const raton = new Producto('Ratón', 20);

alert(teclado);                    // muestra: Teclado (45 €)
console.log(`Producto: ${teclado}`);    // 'Producto: Teclado (45 €)'
console.log('Producto: ' + teclado);    // 'Producto: Teclado (45 €)'
console.log(String(teclado));           // 'Teclado (45 €)'
console.log([teclado, raton].join(' | ')); // 'Teclado (45 €) | Ratón (20 €)'

// También funciona en un objeto literal
const punto = { x: 3, y: 4, toString() { return `(${this.x}, ${this.y})`; } };
console.log(`Punto: ${punto}`);         // 'Punto: (3, 4)'
```

Importante: console.log(teclado) no usa toString(): la consola muestra la estructura del objeto, con sus propiedades, que es lo más útil para depurar (Producto {nombre: 'Teclado', precio: 45}). Para ver en la consola el texto de toString() hay que convertirlo de forma explícita: console.log(String(teclado)) o console.log(`${teclado}`).

Ejercicios · Programación orientada a objetos

Ejercicio 1. Crea una clase llamada Persona. Esta clase deberá tener una propiedad "nombre", de tipo string. También tendrá un método "saludar", que escribirá en la pantalla "Hola, soy " seguido de su nombre. Se crearán dos objetos de tipo Persona, se le asignará un nombre a cada uno y se les pedirá que saluden.

Ejercicio 2. Para guardar información sobre libros, vamos a comenzar por crear una clase "Libro", que contendrá propiedades "autor", "titulo", "ubicacion" (todos ellos strings) y métodos Get y Set adecuados para leer su valor y cambiarlos(el título siempre se guarda en el objeto en mayúsculas y se muestra con *** delante y detrás). Se creará un objeto de la clase Libro **vacío**, le daremos valores a sus tres propiedades (usando los set) y luego los mostrará por pantalla (usando los get) con el formato:
Titulo (Autor) Ubicación

Ejercicio 3. Crear una clase Corredor que tenga las propiedades nombre, dirección, teléfono, país y kilómetros, donde se almacenará el nombre de un usuario, su dirección, teléfono, país de origen (para saber el prefijo a añadir al teléfono +34 en España) y los kilómetros de ejercicio que ha realizado. A la hora de mostrar el teléfono, si el país es España le añadirá +34 y si es Portugal añadirá +35 (solo tendremos en cuenta estos 2 países) hay que tener en cuenta que el país puede tener la primera letra en mayúsculas, todo en mayúsculas… y hay que controlarlo a la hora de añadir +34 o +35 en el teléfono. A la hora de mostrar los kilómetros, éstos se almacenarán en metros por defecto, por lo que si son menores a 1000 los mostrará en metros mostrando: 750m por ejemplo y si son 1500 metros, mostrará: 1,5Km. Se mostrarán los datos en el propio .html, dentro de un div llamado resultado.
Para las pruebas, crear un array con 2 corredores, uno de Portugal y otro de España.

```
let corredores = [
    new Corredor("Manolo", "Ancha 12", "656756463", "España", 23),
    new Corredor("Isa", "San Francisco 5", "666748392", "Portugal", 1320),
```

```
];
```

Realiza pruebas añadiendo otro corredor con el pais España con todo en minúsculas y debe funcionar

7. Estructuras de datos adicionales: Map, Set

7.1 Por qué hacen falta más estructuras

Los objetos literales tienen dos limitaciones que pasan desapercibidas hasta que se necesitan resolver: sus claves solo pueden ser string o symbol (cualquier otra cosa se convierte a texto automáticamente), y no existe una forma directa de saber cuántas propiedades tiene sin pasar por Object.keys(objeto).length.

Los arrays, por su parte, garantizan un orden y permiten duplicados, pero comprobar si contienen un valor concreto (.includes()) obliga a recorrerlos entero en el peor de los casos, y no impiden añadir el mismo valor dos veces si se necesita justo lo contrario: unicidad garantizada.

Map y Set, incorporados en ES6 junto con las clases, están pensados específicamente para estos dos casos.

7.2 Map: pares clave-valor con claves de cualquier tipo

Un Map almacena pares clave-valor, igual que un objeto, pero con una diferencia importante: la clave puede ser de cualquier tipo, incluidos números, booleanos, u otros objetos, sin que se conviertan a texto.

```
const stockPorId = new Map();


stockPorId.set(101, 12); // clave: number
stockPorId.set(102, 0);
stockPorId.set(103, 5);


console.log(stockPorId.get(101)); // 12
console.log(stockPorId.has(102)); // true
console.log(stockPorId.size); // 3 (propiedad directa, sin necesitar Object.keys())


stockPorId.delete(102);
console.log(stockPorId.size); // 2
```

Importante: Con un objeto normal, objeto[101] = 12 convierte la clave 101 a la cadena '101' por debajo, sin avisar. Con un Map, la clave 101 (number) y la clave '101' (string) son dos entradas completamente distintas. Esto también permite usar objetos completos como clave, algo imposible de forma nativa con un objeto literal.

```
const teclado = { nombre: 'Teclado mecánico' };
const raton = { nombre: 'Ratón inalámbrico' };
```

```
const valoracionesPorProducto = new Map();
valoracionesPorProducto.set(teclado, 4.5);
valoracionesPorProducto.set(raton, 4.8);

console.log(valoracionesPorProducto.get(teclado)); // 4.5
```

Modificar el contenido
.set(clave, valor): añade o actualiza un par clave-valor. Si la clave ya existía, sobrescribe su valor. Devuelve el propio Map, lo que permite encadenar varias llamadas seguidas.

```
const stockPorId = new Map();
stockPorId.set(101, 12).set(102, 0).set(103, 5); // encadenado
```

Consultar
.get(clave): devuelve el valor asociado a esa clave, o undefined si no existe.

```
console.log(stockPorId.get(101)); // 12
console.log(stockPorId.get(999)); // undefined
```

.has(clave): comprueba si una clave existe. Devuelve boolean.

```
console.log(stockPorId.has(101)); // true
```

.size: número de pares almacenados (propiedad, sin paréntesis).

```
console.log(stockPorId.size);
```

Recorrer un Map
Un Map mantiene el orden en el que se insertaron sus elementos, y es directamente iterable con for...of, sin pasar por Object.entries() como con un objeto normal.

```
for (const [id, stock] of stockPorId) {
  console.log(`Producto ${id}: ${stock} unidades`);
}

// También se puede convertir a array para usar los métodos del Tema 8
const entradas = [...stockPorId]; // [[101, 12], [103, 5]]
```

Borrado de elementos
Con el método .delete(clave), pasándole la clave del elemento que quieres quitar. Devuelve true si existía y se ha borrado, o false si esa clave no estaba en el Map.

```
const stockPorId = new Map();
stockPorId.set(101, 12);
stockPorId.set(102, 0);

stockPorId.delete(102);
console.log(stockPorId.has(102)); // false
console.log(stockPorId.size);     // 1
```

```
console.log(stockPorId.delete(999)); // false (esa clave no existía)
```

Si en algún momento necesitas vaciarlo por completo, existe también .clear()

7.3 Set: colecciones de valores únicos

Un Set almacena una lista de valores, con una única regla: ningún valor puede repetirse. Si se intenta añadir un valor que ya está, la operación simplemente no hace nada.

```
const categoriasVisitadas = new Set();

categoriasVisitadas.add('Teclados');
categoriasVisitadas.add('Ratones');
categoriasVisitadas.add('Teclados'); // ya existía: no se añade de nuevo

console.log(categoriasVisitadas.size); // 2, no 3
console.log(categoriasVisitadas.has('Ratones')); // true
```

Modificar el contenido
.add(valor): añade un valor. Si ya existe, no hace nada (sin error, sin duplicar).

```
const categorias = new Set();
categorias.add('Teclados');
categorias.add('Ratones');
categorias.add('Teclados'); // no se añade de nuevo
```

.delete(valor): elimina un valor concreto. Devuelve true si existía y se ha borrado, false si no estaba (mismo comportamiento que Map.delete()).

```
categorias.delete('Ratones');
console.log(categorias.delete('Monitores')); // false, no existía
```

.clear(): vacía el Set por completo.

```
categorias.clear();
console.log(categorias.size); // 0
```

Consultar
.has(valor): comprueba si un valor está presente. Devuelve boolean.

```
console.log(categorias.has('Teclados')); // true o false, según lo que quede
```

.size: número de elementos (es una propiedad, no un método: sin paréntesis, como .length en un array pero sin necesitar contar nada a mano).

```
console.log(categorias.size);
```

Recorrer
for...of: la forma habitual de iterar, en el mismo orden en que se insertaron los valores.

```
const idsUnicos = new Set([101, 102, 103]);
```

```
for (const id of idsUnicos) {
  console.log(id);
}
```

Conversión a array con spread

```
const array = [...idsUnicos]; // [101, 102, 103]
```

Así puedo utilizar todos los métodos de Array.

El caso de uso más habitual: eliminar duplicados de un array

Combinado con el operador *spread*, un Set es la forma más directa de quitar duplicados de un array.

```
const idsConDuplicados = [101, 102, 101, 103, 102, 104];

const idsUnicos = [...new Set(idsConDuplicados)];
console.log(idsUnicos); // [101, 102, 103, 104]
```

8. Módulos en JavaScript

Hasta ahora, todo el código de un ejemplo cabía en un único fichero. Un proyecto real se divide en varios ficheros que se comunican entre sí mediante export e import.

8.1 Por qué dividir el código en módulos

Cuando todo el código de una página se escribe en un único fichero .js (o en varios ficheros cargados con etiquetas <script> normales), todas las variables y funciones declaradas de esa forma comparten el mismo espacio. Esto genera dos problemas al crecer el proyecto.

- **Colisión de nombres**: si dos ficheros distintos declaran una función o variable con el mismo nombre en el ámbito global, uno sobrescribe al otro, sin ningún aviso.
- **Orden de carga manual**: si un script depende de una función definida en otro, hay que asegurarse a mano de que las etiquetas <script> aparezcan en el orden correcto en el HTML, algo frágil y fácil de romper al añadir más ficheros.

Un módulo es, simplemente, un fichero .js con su propio ámbito: nada de lo que se declara dentro es visible desde fuera, a menos que se exporte de forma explícita. Y nada de lo que hay en otro fichero es visible dentro, a menos que se importe de forma explícita.

8.2 export: exportar código desde un módulo

La palabra clave export, delante de una declaración, la hace accesible desde otros módulos. Existen dos formas: exportaciones nombradas y exportación por defecto.

Exportaciones nombradas

Un módulo puede tener tantas exportaciones nombradas como se necesite. Se pueden exportar en el momento de declarar, o agruparlas al final del fichero.

```
export const TIPO_IVA_GENERAL = 0.21;
```

```
export function calcularPrecioConIva(precioBase, tipoIva = TIPO_IVA_GENERAL) {
  return Number((precioBase * (1 + tipoIva)).toFixed(2));
}
```

Alternativa, exportando todo junto al final del fichero (mismo resultado):

```
const TIPO_IVA_GENERAL = 0.21;

function calcularPrecioConIva(precioBase, tipoIva = TIPO_IVA_GENERAL) {
  return Number((precioBase * (1 + tipoIva)).toFixed(2));
}

export { TIPO_IVA_GENERAL, calcularPrecioConIva };
```

Exportación por defecto

Un módulo puede tener, como mucho, **una** exportación default: pensada para "lo principal" que ofrece ese fichero (una clase, una función, un objeto de configuración).

```
export default class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }
}
```

Nota: Como norma orientativa: usa export default cuando el fichero tiene un único propósito principal y claro (una clase, un componente). Usa exportaciones nombradas cuando el fichero reúne varias utilidades relacionadas pero independientes entre sí (como iva.js en el ejemplo anterior), de forma que quien importe pueda coger solo lo que necesita.

8.3 import: importar código en otro módulo

Importar exportaciones nombradas

Se importan entre llaves, con el mismo nombre que se les dio al exportarlas.

```
import { TIPO_IVA_GENERAL, calcularPrecioConIva } from './iva.js';

console.log(calcularPrecioConIva(100)); // 121
console.log(TIPO_IVA_GENERAL); // 0.21
```

Importar la exportación por defecto

Se importa sin llaves, y se le puede dar cualquier nombre al importarla (no tiene por qué coincidir con el nombre original).

```
import Producto from './producto.js';

const teclado = new Producto('Teclado', 45);
```

Combinar ambas, y renombrar con as

```
import Producto, { TIPO_IVA_GENERAL, calcularPrecioConIva as calcularConIva } from './iva.js';


console.log(calcularConIva(100)); // se usa con el nombre nuevo, "calcularConIva"
```

8.4 Módulos en el navegador: type="module"

Para que un navegador trate un fichero como módulo (y permita usar export/import en él), el <script> que lo carga debe llevar el atributo type="module".

```
<script type="module" src="./app.js"></script>
```

Cargar un script como módulo cambia varias cosas respecto a un script normal:

- Se comporta como si llevara defer de forma automática: se descarga en paralelo y se ejecuta cuando el HTML ya está listo.
- Se ejecuta siempre en modo estricto, sin necesidad de indicarlo explícitamente.
- Tiene su propio ámbito: las variables de nivel superior de un módulo no pasan a formar parte de window, a diferencia de lo que ocurre con un script normal.

8.5 Organizar un proyecto en varios módulos

Un pequeño proyecto puede organizarse separando responsabilidades en distintos ficheros, en lugar de amontonar todo en uno solo.

producto.js

```
export default class Producto {
  constructor(nombre, precio, stock) {
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
  }
}
```

inventario.js

```
export function calcularValorInventario(productos) {
  return productos
    .filter(producto => producto.stock > 0)
    .reduce((total, producto) => total + producto.precio * producto.stock, 0);
}


export function productosAgotados(productos) {
  return productos.filter(producto => producto.stock === 0);
}
```

app.js

```
import Producto from './producto.js';
```

```
import { calcularValorInventario, productosAgotados } from './inventario.js';

const productos = [
  new Producto('Teclado', 45, 12),
  new Producto('Ratón', 20, 0),
  new Producto('Monitor', 150, 5)
];

console.log(calcularValorInventario(productos)); // 45*12 + 150*5 = 1290
console.log(productosAgotados(productos)); // [Producto { nombre: 'Ratón', ... }]
```

Nota: Cada fichero tiene una responsabilidad clara: **producto.js** define la forma de un producto, **inventario.js** agrupa cálculos sobre colecciones de productos, y **app.js** los combina. Esta forma de organizar el código (ficheros pequeños, cada uno con un propósito concreto, comunicados mediante import/export) es exactamente la misma que se usará al construir componentes con React.